

Основы Django

Взаимодействие с базой данных через Django ORM



На этом уроке

1. Познакомимся с Django ORM.
2. Разберём процесс создания моделей.
3. Поработаем с Django ORM из консоли.
4. Научимся заносить данные в базу.
5. Узнаем, как отображать данные из базы.

Оглавление

[Django ORM. Введение](#)

[Подключение проекта к базе данных](#)

[SQLite](#)

[Инструменты для работы с базами данных](#)

[Модели](#)

[Типы полей](#)

[Первичный ключ](#)

[Принцип «толстых моделей»](#)

[Поля связи](#)

[Связь «один ко многим»](#)

[Связь «многие ко многим»](#)

[Каскадное удаление данных](#)

[Класс Meta](#)

[Механизм миграций](#)

[Подключение к базе данных](#)

[Именованное таблицы](#)

[Использование моделей](#)

[Консоль проекта](#)

[Создание объектов](#)

[Получение объектов](#)

[Выборка всех объектов](#)

[Фильтрация](#)

[Исключение объектов](#)

[Получение списка значений](#)

[Выборка конкретных объектов](#)

[Обновление данных в объектах](#)

[Обновление через свойство полученного объекта](#)

[Обновление объекта через метод менеджера модели](#)

[Удаление объектов](#)

[Заполнение базы данными](#)

[Фикстуры](#)

[Создание фикстуры](#)

[Загрузка фикстуры](#)

[Преимущества и недостатки фикстур](#)

[Миграции данных](#)

[Откат миграций](#)

[Решение типовых проблем учебного проекта](#)

[Бинарные данные](#)

[Вывод данных из базы на страницы](#)

[Автоматическая фильтрация удалённых объектов](#)

[Заключение](#)

[Практическое задание](#)

[Глоссарий](#)

[Дополнительные материалы](#)

[Приложение А. Создание моделей](#)

[Приложение Б. Фикстуры](#)

[Фикстура курсов](#)

[Фикстура уроков](#)

[Фикстура преподавателей](#)

[Приложение В. Вывод данных из базы на страницы](#)

[Модели](#)

[Диспетчер адресов](#)

[Контроллеры](#)

[Шаблон страницы новостей](#)

[Шаблон списка курсов](#)

[Шаблон детального отображения курса](#)

[Шаблон детального отображения новости](#)

[Базовый шаблон](#)

[Изображение по умолчанию](#)

Django ORM. Введение

Если веб-проект представляет собой полноценный веб-ресурс с отображением множества динамичных данных, то они будут храниться в реляционной базе. Системы управления базами данных, а именно MySQL, PostgreSQL и прочие, обычно сопровождаются системой языка структурированных запросов:

- S — structured, структурированный;
- Q — query, запрос;
- L — language, язык.

Это позволяет извлекать данные из таблиц. Такие таблицы представляют собой наборы строк и столбцов (далее — полей).



Рис. 1. Визуальное представление СУБД и ORM

Поля таблицы описывают содержимое конкретного поля. Такое описание очень похоже на свойство класса в ООП. В рамках этого соотношения реализуются такие специальные системы, как ORM (Object-Relational Mapping) — объектно-реляционное отображение. Они предоставляют разработчику возможность абстрагироваться от языка запросов (SQL) и сосредоточиться на описании взаимодействия объектов, используя код, пока система ORM общается с базами данных.

Некоторые преимущества такого разделения ответственности:

1. Взаимодействие с объектами базы данных как с обычными объектами.
2. Абстракция от базы данных и возможность использования одного и того же программного интерфейса во всех поддерживаемых СУБД.
3. Нет дополнительного языка в кодовой базе.

4. Простой перенос структуры объектов из проекта в проект путём миграций.

Однако у ORM-систем есть и недостатки:

1. Универсализация решений взаимодействия с базами данных даёт избыточность в запросах, поскольку для точного результата выборки требуется максимально широко описать запрос.
2. На большие запросы тратится много времени, что критично для нагруженных систем.
3. Универсальный программный интерфейс часто ведёт к ограниченной функциональности взаимодействия с СУБД.

Взаимодействие с базой данных через системы ORM описывает фраза: «Выигрываем в удобстве, проигрываем в быстродействии».

Поскольку Django — это фреймворк, где большинство необходимых средств доступно «из коробки», ORM также вошёл в состав.

Подключение проекта к базе данных

Все дальнейшие примеры основываются на учебном проекте LMS Braniac.

Настройки содержатся в главном конфигурационном файле `config/settings.py`, в переменной `DATABASES`.

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Поскольку проекты взаимодействуют с множеством баз данных, то первый ключ словаря `default` обозначает условное имя БД. Количество баз, с которыми происходит взаимодействие, не ограничено. В качестве значения ставится ещё один словарь, где описываются параметры подключения. В нашем случае таких параметров два:

1. `ENGINE` — движок, на основе которого происходит взаимодействие с БД. Django «из коробки» поддерживает СУБД PostgreSQL, MySQL/MariaDB, SQLite и Oracle.
2. `NAME` — имя базы данных.

Настроек для базы данных при использовании СУБД может быть значительно больше. Подробнее об этом — в дополнительных материалах.

Так выглядит настройка подключения к СУБД PostgreSQL:

```
DATABASES = {
```

```
'default': {  
    'ENGINE': 'django.db.backends.postgresql',  
    'NAME': 'mydatabase',  
    'USER': 'mydatabaseuser',  
    'PASSWORD': 'mypassword',  
    'HOST': '127.0.0.1',  
    'PORT': '5432',  
}
```

В настройках указываются всё необходимое для подключения к серверу БД: имя базы (NAME), имя пользователя (USER), пароль (PASSWORD), адрес расположения сервера (HOST), порт подключения (PORT).

Больше об этом — в разделе [дополнительных материалов](#).

SQLite

На этапе разработки на учебном проекте мы будем использовать СУБД SQLite. Она очень удобна для локальной разработки, потому что представляет собой обычный файл и распространяется под лицензией общественного достояния. Это значит, что кто угодно может использовать эту СУБД в любых своих целях.

С SQLite легко настроить сервер. В этом заключается её преимущество и недостаток, поскольку SQLite представляет собой один файл, доступ к ней может получить один пользователь за один раз. Эту БД активно используют разработчики для создания мобильных приложений, когда надо хранить пользовательские данные в структурированном виде. Для реальных веб-серверных приложений она не подходит, потому что для этого требуются тысячи одновременных подключений.

Инструменты для работы с базами данных

Для взаимодействия с файлом базы данных используются:

1. [SQLiteStudio](#) — кросс-платформенная программа, которая распространяется под свободной лицензией. Она сосредоточена на поддержке конкретной СУБД и поддерживает весь спектр возможных операций.
2. [DBeaver](#) — кросс-платформенная программа для работы с различными СУБД. Как и PyCharm встречается в двух редакциях: Community и Enterprise. Она поддерживает множество самых распространённых СУБД: PostgreSQL, SQLite, MySQL, MariaDB и т. д. Community-редакция распространяется по открытой и бесплатной лицензии.
3. В [PyCharm Professional](#) также есть встроенная поддержка работы с различными СУБД.

4. В VSCode есть много [плагинов](#), которые позволяют взаимодействовать с разными типами СУБД.
5. Встроенный в django механизм подключения к БД с использованием командной строки.

Модели

Концепция ORM позволяет переносить представления из таблиц БД в классы ООП. В Django есть классы, которые описывают структуру таблиц. Они называются *моделями*. Модели — это последняя структурная единица паттерна MVT (**M** — models). Чтобы описать поля таблиц в БД, надо задать свойства класса.

Рассмотрим модели в Django ORM подробнее. В учебной системе Braniac есть страничка новостей.

Так выглядит модель, описывающая таблицу с новостями:

mainapp/models.py

```
from django.db import models

class News(models.Model):
    title = models.CharField(max_length=256, verbose_name="Title")
    preamble = models.CharField(max_length=1024, verbose_name="Preamble")
    body = models.TextField(blank=True, null=True, verbose_name="Body")
    body_as_markdown = models.BooleanField(
        default=False, verbose_name="As markdown"
    )
    created = models.DateTimeField(
        auto_now_add=True, verbose_name="Created", editable=False
    )
    updated = models.DateTimeField(
        auto_now=True, verbose_name="Edited", editable=False
    )
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return f"{self.pk} {self.title}"
```

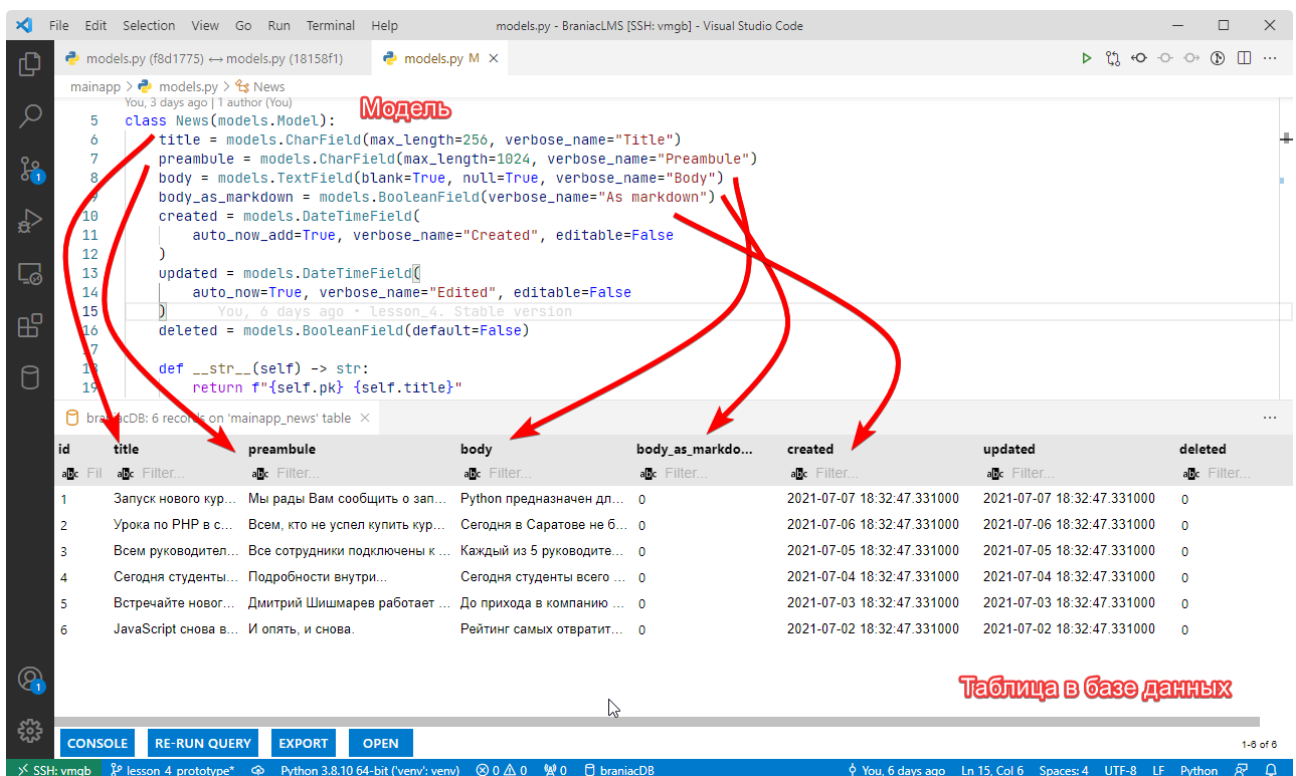



Рис. 2. Модель, описывающая таблицу в базе данных

Типы полей

Следуя принципу ООП текущий класс модели наследуется от класса `models.Model`, который определяет основное поведение моделей в рамках Django ORM. В модуле `models` также есть набор классов, который позволяет задать типы полей.

Например, первое свойство класса `News` — `title` описывает поле, где будет храниться заголовок новостей. Это происходит путём использования класса `CharField`, у которого есть один обязательный именованный параметр `max_length`, определяющий максимальную длину текста в поле.

Ещё один именованный параметр — `verbose_name`. Он создаёт описание, которое, например, используется для обозначения полей формы.

В Django есть классы типов полей, предназначенные для большинства задач. Их можно найти в документации. В документации также есть описание специфичных полей, которые характерны для конкретной СУБД. Все необходимые ссылки — в [дополнительных материалах](#).

В описании поля `body` есть связка именованных аргументов `blank=True`, `null=True`. Параметр `blank` связан с проверкой значения, которое вводится в форме, связанной с этой моделью. Если выставлено значение `True`, то при заполнении формы его можно не заполнять. А `null` связан с занесением значения в таблицу БД. Если параметр выставлен в `True`, то в поле пустое значение выставится как `NULL`.

У следующего поля `body_as_markdown` тип `BooleanField`. Для этого поля не заданы параметры пустого значения, и по сути, оно имеет всего два состояния: `True` или `False`. Значит, для него задаётся именованный необязательный аргумент `default=False`. Это поле определяет свойство отображения текста из `body`. При установке значения в `True` значение поля `body` дополнительно обрабатывается как размеченное в формате `markdown`. Соответственно, если нет дополнительного указания значения поля, оно (поле) по умолчанию выставляет значение `False`.

Следующие два поля — `created` и `updated`. Их значения — это дата и время создания и обновления объекта. Есть отдельный паттерн создания моделей, при котором описывается отдельная модель, только с этими двумя полями, а все последующие модели наследуются от неё. Таким образом, для этого в БД создаётся отдельная таблица. Этот подход удобен, но при больших объёмах базы данных он увеличивает время выборки.

В описанных свойствах есть два параметра, которые надо рассмотреть отдельно. Параметр `auto_now_add=True` говорит, что значение в нём установится автоматически при создании записи в БД. Он отлично подходит для поля `created` и полностью определяет его поведение. С другой стороны, в поле `updated` выставлен параметр `auto_now=True`. Этот параметр говорит, что значение поля будет обновляться при каждом изменении записи в БД (обновлении объекта модели).

У поля `deleted` указан уже знакомый тип `BooleanField`, а значение по умолчанию — `default=False`. Оно содержит признак того, что объект удалён. Это стандартный паттерн: любая занесённая в базу информация может помечаться как удалённая, но не быть таковой. Простой пример: удаление профиля в социальных сетях. Практически все компании предоставляют возможность в некоторый срок восстановить профиль. Если компания добросовестная, то по окончании указанного периода информация действительно удаляется. Однако обычно этого не происходит, потому что любые данные можно проанализировать и на основе этого улучшить продукт.

Первичный ключ

В этой модели нет отдельного поля уникального идентификатора. Обычно в БД за него отвечает поле `id`. Поскольку это популярный паттерн использования такого поля, в рамках Django он реализуется по умолчанию. То есть внутри Django поле `id` автоматически определяется для всех моделей, унаследованных от стандартного класса.

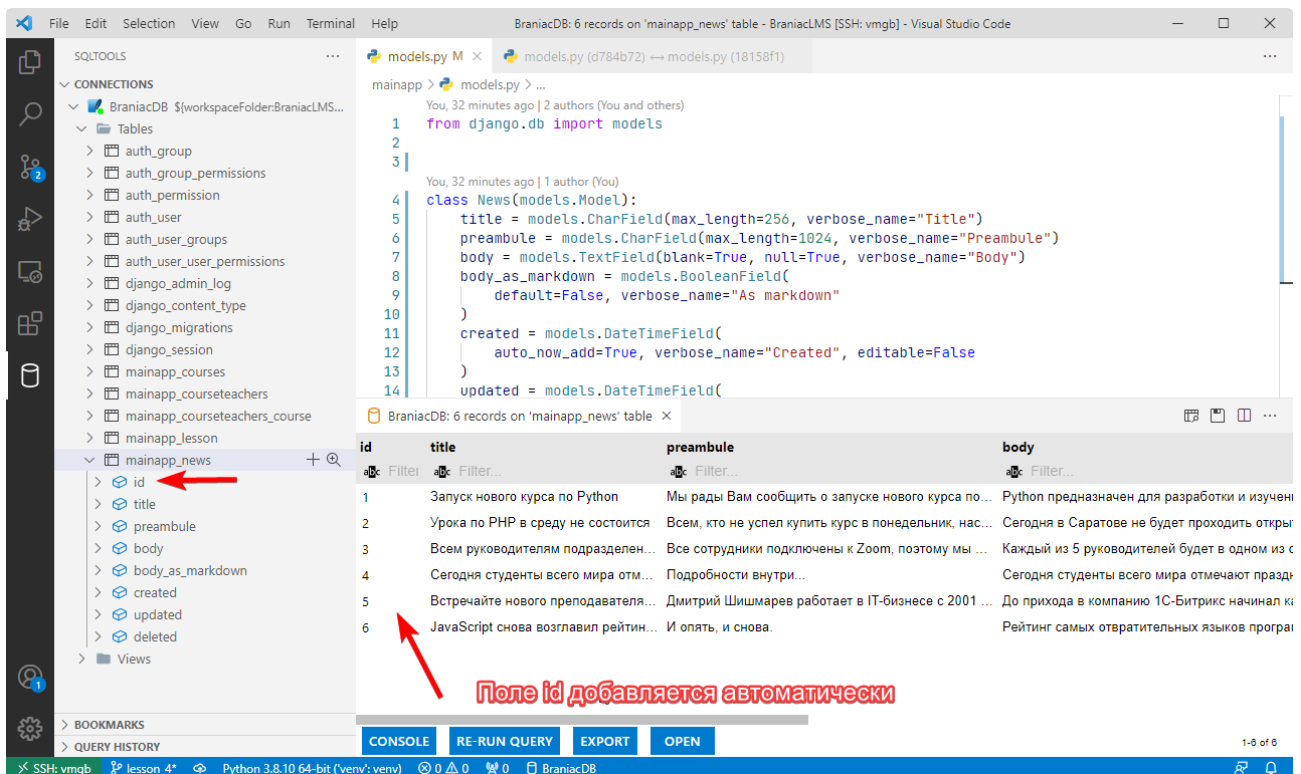


Рис. 3. Автоматическое добавление поля id для всех моделей по умолчанию

Полю по умолчанию присваивается признак первичного ключа. Это можно изменить, чтобы любое поле стало первичным ключом. Для этого достаточно указать в параметрах поля именованный аргумент `primary_key=True`.

Мы не меняли поведение по умолчанию в этой модели. Поэтому при выборке из базы, указывая фильтрации по id или pk (primary key), эти фильтрации будут равносильны.

Принцип «толстых моделей»

Принцип толстых моделей — это конвенция Django. Согласно ей, все операции, связанные с объектами, размещаются в рамках модели. Поскольку модели — это классы Python, все операции с объектами размещаются в методах класса модели.

Один из самых часто определяемых методов для модели — строковое представление объектов через метод `__str__`.

Поскольку в модели для объектов выставляется признак удаления, надо реализовать метод, который позволит это сделать.

После внесения изменений код модели будет выглядеть так:

```
mainapp/models.py

class News(models.Model):
    title = models.CharField(max_length=256, verbose_name="Title")
```

```
preamble = models.CharField(max_length=1024, verbose_name="Preamble")
body = models.TextField(blank=True, null=True, verbose_name="Body")
body_as_markdown = models.BooleanField(
    default=False, verbose_name="As markdown"
)
created = models.DateTimeField(
    auto_now_add=True, verbose_name="Created", editable=False
)
updated = models.DateTimeField(
    auto_now=True, verbose_name="Edited", editable=False
)
deleted = models.BooleanField(default=False)

def __str__(self) -> str:
    return f"{self.pk} {self.title}"

def delete(self, *args):
    self.deleted = True
    self.save()
```

Метод delete обеспечит установку свойства с последующим сохранением объекта.

Поля связи

Связь «один ко многим»

Чтобы не перегружать таблицы для БД, в них добавляются поля со ссылками на другие таблицы.


```

deleted = models.BooleanField(default=False)

def __str__(self) -> str:
    return f"{self.pk} {self.name}"

def delete(self, *args):
    self.deleted = True
    self.save()

class Lesson(models.Model):
    course = models.ForeignKey(Courses, on_delete=models.CASCADE)
    num = models.PositiveIntegerField(verbose_name="Lesson number")
    title = models.CharField(max_length=256, verbose_name="Name")
    description = models.TextField(
        verbose_name="Description", blank=True, null=True
    )
    description_as_markdown = models.BooleanField(
        verbose_name="As markdown", default=False
    )
    created = models.DateTimeField(
        auto_now_add=True, verbose_name="Created", editable=False
    )
    updated = models.DateTimeField(
        auto_now=True, verbose_name="Edited", editable=False
    )
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return f"{self.course.name} | {self.num} | {self.title}"

    def delete(self, *args):
        self.deleted = True
        self.save()

    class Meta:
        ordering = ("course", "num")

```

В модели урока Lesson первым идёт поле course, типа ForeignKey. В качестве первого аргумента этот тип поля принимает имя класса, на который сформируется внешняя ссылка. В таблице появится поле, состоящее из имени, указанного в модели, и суффикса `_id`. В нашем случае это будет поле с именем

course_id, содержащее значение id той таблицы, на которую происходит ссылка — id из таблицы с курсами.

Связь «многие ко многим»

Ещё один вид связи в БД — «многие ко многим». Например, у одной книги — несколько авторов, но каждый из них работает над множеством книг. Рассмотрим эту связь на примере преподавателей: разные преподаватели ведут разные курсы.

Вид модели:

mainapp/models.py

```
class CourseTeachers(models.Model):
    course = models.ManyToManyField(Courses)
    name_first = models.CharField(max_length=128, verbose_name="Name")
    name_second = models.CharField(max_length=128, verbose_name="Surname")
    day_birth = models.DateField(verbose_name="Birth date")
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return "{0:0>3} {1} {2}".format(
            self.pk, self.name_second, self.name_first
        )

    def delete(self, *args):
        self.deleted = True
        self.save()
```

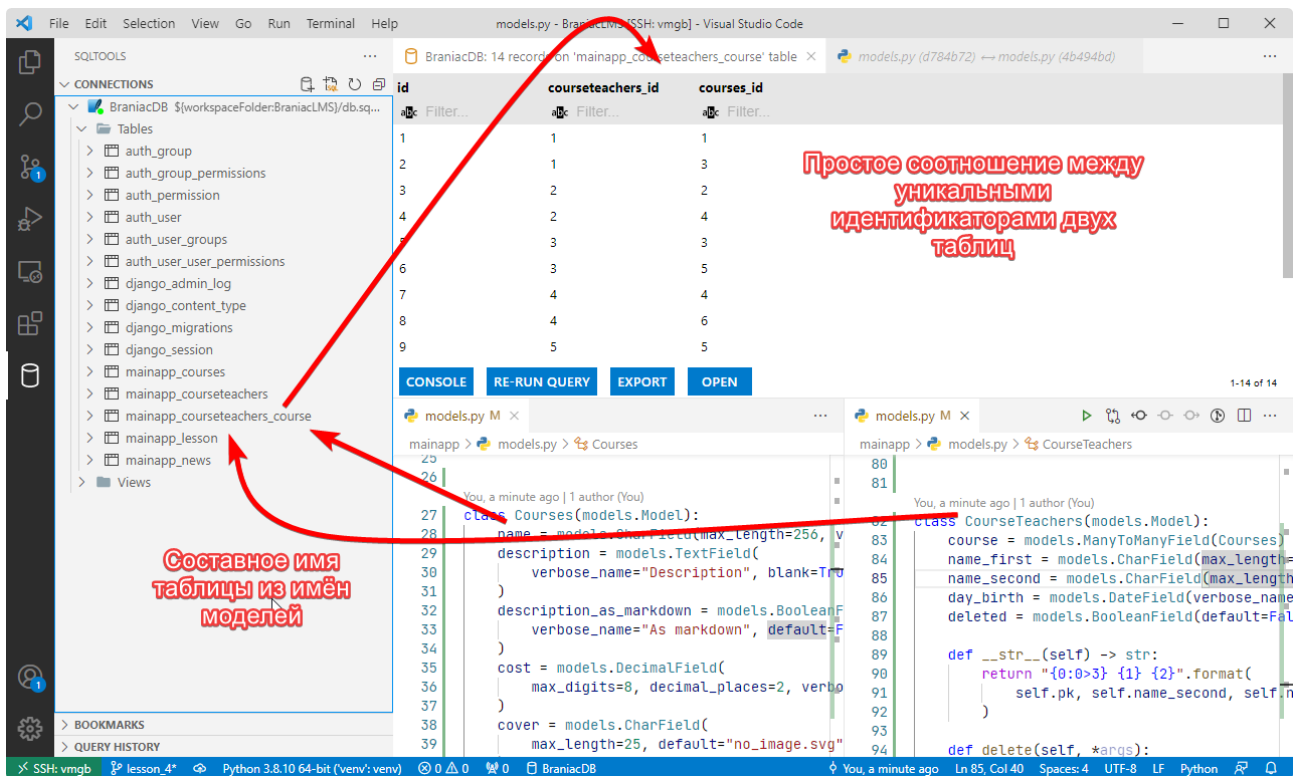


Рис. 5. Связь «многие ко многим», с точки зрения БД

При описании этой связи указывается тип поля `models.ManyToManyField`. А в качестве первого позиционного аргумента передаётся класс модели, на которую создаётся внешняя связь.

С точки зрения БД, эта связь представляет собой создание отдельной таблицы, где соотносятся уникальные идентификаторы каждой таблицы.

Каскадное удаление данных

Второй именованный аргумент, который передаётся в тип поля `ForeignKey` — указание, как поступать с элементами внешней связи. При установке аргумента `on_delete=models.CASCADE` для текущего поля сформируется правило. Согласно ему, при удалении элемента текущий элемент также удалится. Например, при удалении курса удалятся и все связанные уроки.

При удалении внешней связи устанавливается значение `NULL`. Для этого надо задать параметр `on_delete=models.SET_NULL`, а также обозначить, что поле устанавливается в значении `null=True`.

Класс Meta

Класс `Meta` в моделях Django — это своеобразная конвенция, куда заносятся все дополнительные данные, связанные с моделью.

В модели уроков `Lesson` в классе `Meta` есть порядок сортировки: `ordering = ("course", "num")`. Это значит, что при выборке объектов потребуется отсортировать их сначала по полю `course`, а затем — по полю `num`.

Один из популярных приёмов этого класса — обозначение имени выводимых объектов в единственном (параметр `verbose_name`) и во множественном числе (параметр `verbose_name_plural`).

Полный получившийся код моделей представлен в [приложении А](#).

Механизм миграций

После написания классов моделей надо перенести созданную структуру в БД. Для этого применяется *механизм миграций*.

С точки зрения Django, понятие *миграции* представляется в двух вариантах:

1. Миграция — *процесс* переноса созданных изменений в базу данных.
2. Миграция — *файл*, описывающий изменения, которые надо внести в базу данных.

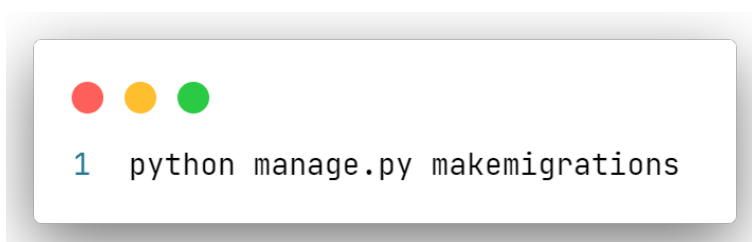
При работе с механизмом миграций важно соблюдать последовательность действий:

1. Создание файлов миграций.
2. Выполнение процедуры миграций.

```
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py makemigrations
Migrations for 'mainapp':
  mainapp/migrations/0001_initial.py
    - Create model Courses
    - Create model News
    - Create model Lesson
    - Create model CourseTeachers
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$
```

Рис. 6. Процесс создания миграций

Для создания миграций надо выполнить команду:



Без дополнительных параметров эта команда:

1. Просматривает изменения каждой модели зарегистрированных приложений в переменной `INSTALLED_APPS` главного конфигурационного файла проекта.
2. Для всех найденных изменений создаёт файлы миграций, которые будут размещаться по пути: `<имя приложения>/migrations/`.

Команда запускается с дополнительными параметрами:

1. `--merge` — ключ, который позволяет разрешать конфликты внесения изменений в базу. В основном используется при совместной работе над одной веткой проекта. После запуска команды с этим ключом сформируется дополнительная миграция.
2. `--empty` — ключ, который позволяет создавать файл с пустой миграцией.
3. `--name NAME` — ключ и значение, которые позволяют задать имя файла миграции. С ними дополнительно задаётся имя миграции, чтобы обозначить значимость и/или быстрый доступ.
4. `app_label` — основной позиционный параметр, который вводится после всех ключей. Он задаёт имя приложения, для которого требуется создать файлы миграций. Удобен, когда надо внести изменения в модели разных приложений, а работы над некоторыми изменениями в других приложениях ещё незакончены.

Например, эта команда создаст пустую миграцию с именем `newDataToTable` для приложения `mainapp`, но сейчас её запускать не надо:



```
1 python manage.py makemigrations --empty --name newDataToTable mainapp
```

Внутри папок приложений папка, содержащая файлы миграций `migrations`, представляет собой Python-пакет:

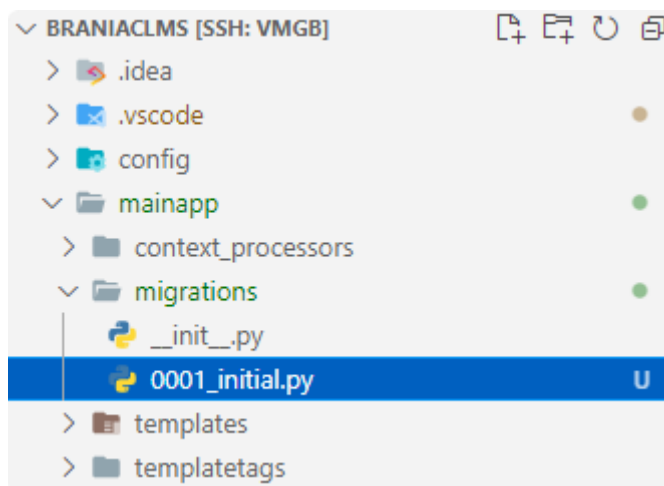


Рис. 7. Файл миграции в структуре проекта

Пока у нас в проекте всего одно приложение, поэтому после выполнения команды в папке `mainapp/migrations` появится новый файл с именем `0001_initial.py`. У него есть числовой суффикс, который обозначает номер миграции. Все миграции выполняются последовательно.

Важно соблюдать эти правила:

1. Изменять имена миграций нельзя. Миграции — это последовательность изменений базы данных, и если изменить имя файла, то при проверке целостности внесения изменений система даст сбой. Это грозит потерей данных или повреждением структуры БД.
2. Нельзя изменять содержимое файла миграций. Каждый файл миграций содержит изменения, которые вносятся в структуру БД. Изменив файл миграции, можно повредить целостность структуры БД. Из этого правила есть одно исключение — миграции данных, о которых речь пойдёт далее.

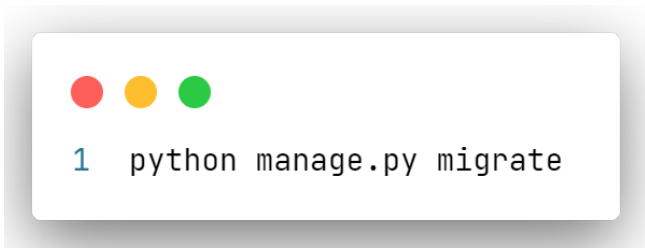
Поскольку проекты и структуры моделей постоянно меняются, эти изменения надо переносить и в базу данных. Файлы миграций отражают суть этих изменений. Выполняя перенос изменений в моделях на базу данных, система последовательно проходит по цепочке файлов миграций.

Ещё одно важное правило при работе с механизмом миграции и Django ORM: при **любых** изменениях в моделях надо создать файлы миграций и провести процедуру миграции.

```
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, mainapp, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying mainapp.0001_initial... OK
  Applying sessions.0001_initial... OK
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$
```

Рис. 8. Процесс применения миграций

Для применения миграций выполняется команда:

A terminal window with a white background and a grey border. At the top, there are three colored circles: red, yellow, and green. Below them, the text '1 python manage.py migrate' is displayed in a monospaced font.

```
1 python manage.py migrate
```

Произойдёт последовательное применение файлов миграций. При успешном применении миграции в статусе будет ОК. Если в процессе проведения процедуры на каком-то этапе произойдёт ошибка, то процесс прервётся.

Подключение к базе данных

При работе над проектом мы рекомендуем иметь быстрый доступ к базе данных через какое-либо [программное обеспечение](#). Это позволит в реальном времени отслеживать, как изменяется база и данные в ней. Используя некоторые инструменты (например, в DBeaver), можно быстро заполнить базу отладочными данными через генератор данных.

```
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py dbshell
SQLite version 3.31.1 2020-01-27 19:55:54
Enter ".help" for usage hints.
sqlite> .tables
auth_group          django_migrations
auth_group_permissions  django_session
auth_permission     mainapp_courses
auth_user           mainapp_courseteachers
auth_user_groups    mainapp_courseteachers_course
auth_user_user_permissions  mainapp_lesson
django_admin_log    mainapp_news
django_content_type
sqlite> █
```

Рис. 9. Подключение к командной оболочке SQLite

cid	name	type	notnull	dfit_value	pk
0	id	integer	1	NULL	1
1	title	varchar(256)	1	NULL	0
2	preamble	varchar(1024)	1	NULL	0
3	body	text	0	NULL	0
4	body_as_markdown	bool	1	NULL	0
5	created	datetime	1	NULL	0
6	updated	datetime	1	NULL	0
7	deleted	bool	1	NULL	0

Рис. 10. Свойства таблицы в расширении SQLTools для VSCode

Вместе с Django «из коробки» предоставляется команда, которая позволяет автоматически подключаться к командной оболочке базы данных, если она предусмотрена. Так выглядит эта команда:

```

1 # Подключение к командной оболочке БД
2 python manage.py dbshell

```

Она выполняет проброс запуска необходимого программного обеспечения через командную строку. Проверьте, что у вас установлено соответствующее ПО. Если его нет, программа выдаст ошибку.

Работая с базой данных через командную оболочку, надо учитывать особенности SQL-запросов для соответствующей СУБД. Ссылки на документации для наиболее распространённых СУБД есть в дополнительных материалах.

Именованние таблиц

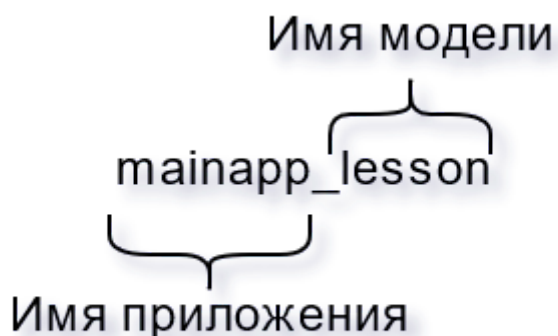


Рис. 11. Наименование таблицы в БД по имени модели

Подключившись к базе данных, можно заметить особенность именования таблиц: они состояются из имени приложения и модели в нижнем регистре через символ подчёркивания.

Использование моделей

Параллельно откроем любое удобное средство просмотра содержимого БД.

Консоль проекта

Помимо подключения к командной оболочке базы данных проекта, есть отдельная команда подключения к интерактивному интерпретатору Python.

```
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py shell
Python 3.8.10 (default, Jun 2 2021, 10:49:15)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from mainapp.models import News
>>> █
```

Рис. 12. Интерактивный интерпретатор Python, подключённый к проекту

Команда выглядит так:



```
1 # Подключение к интерактивному
2 # интерпретатору Python
3 python manage.py shell
```

```
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python
Python 3.8.10 (default, Jun 2 2021, 10:49:15)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from mainapp.models import News
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/mate/Desktop/djangoBasics/BraniacLMS/mainapp/models.py", line 4, in <module>
    class News(models.Model):
  File "/home/mate/Desktop/djangoBasics/venv/lib/python3.8/site-packages/django/db/models/base.py", line 108, in __new__
    app_config = apps.get_containing_app_config(module)
  File "/home/mate/Desktop/djangoBasics/venv/lib/python3.8/site-packages/django/apps/registry.py", line 250, in get_containing_app_config
    self.check_apps_ready()
  File "/home/mate/Desktop/djangoBasics/venv/lib/python3.8/site-packages/django/apps/registry.py", line 150, in check_apps_ready
    settings.INSTALLED_APPS
  File "/home/mate/Desktop/djangoBasics/venv/lib/python3.8/site-packages/django/conf/__init__.py", line 201, in __getattr__
    self._setup(name)
  File "/home/mate/Desktop/djangoBasics/venv/lib/python3.8/site-packages/django/conf/__init__.py", line 184, in _setup
    raise ImproperlyConfigured(
django.core.exceptions.ImproperlyConfigured: Requested setting INSTALLED_APPS, but settings
DJANGO_SETTINGS_MODULE or call settings.configure() before accessing settings.
>>> █
```

Рис. 13. Ошибка при попытке импорта объектов проекта из неподключенного к проекту интерактивного интерпретатора Python

Чтобы взаимодействовать с объектами проекта, надо подключаться через указанную команду.

Умение пользоваться этим инструментом выручает при процессе отладки. Например, когда происходит остановка исполнения на точке остановки (*breakpoint*), в любых IDE есть возможность воспользоваться консолью отладки. Опционально она аналогична интерактивному интерпретатору, подключённому к проекту.

Создание объектов

После выполнения процедуры миграции в новом проекте все таблицы сформировались, но без данных. Чтобы создать объект, в интерактивном интерпретаторе Python требуется импортировать соответствующий класс. В качестве примера возьмём класс модели новостей `mainapp.models.News`.

Для импорта класса введём команду:

```

1 # Импорт класса модели новостей
2 from mainapp.models import News

```

Объект формируется двумя способами:

1. Создать объект класса модели и сохранить его.
2. Воспользоваться методом *менеджера модели*.

Создание объекта:

```

1 # Создание объекта Python
2 news_obj = News(title="News001", preamble="SomeNewNews001", body="News001Body")

```

Запись в БД не сформируется. Для сохранения изменений надо вызвать метод `.save()`.

При создании простого объекта запись в базу данных попадёт только после вызова метода `.save()`

```

(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py shell
Python 3.8.10 (default, Jun  2 2021, 10:49:15)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from mainapp.models import News
>>> news_obj = News(title="News001", preamble="SomeNewNews001", body="News001Body")
>>> news_obj.save()
>>>

```

Рис. 14. Сохранение объекта как записи в БД

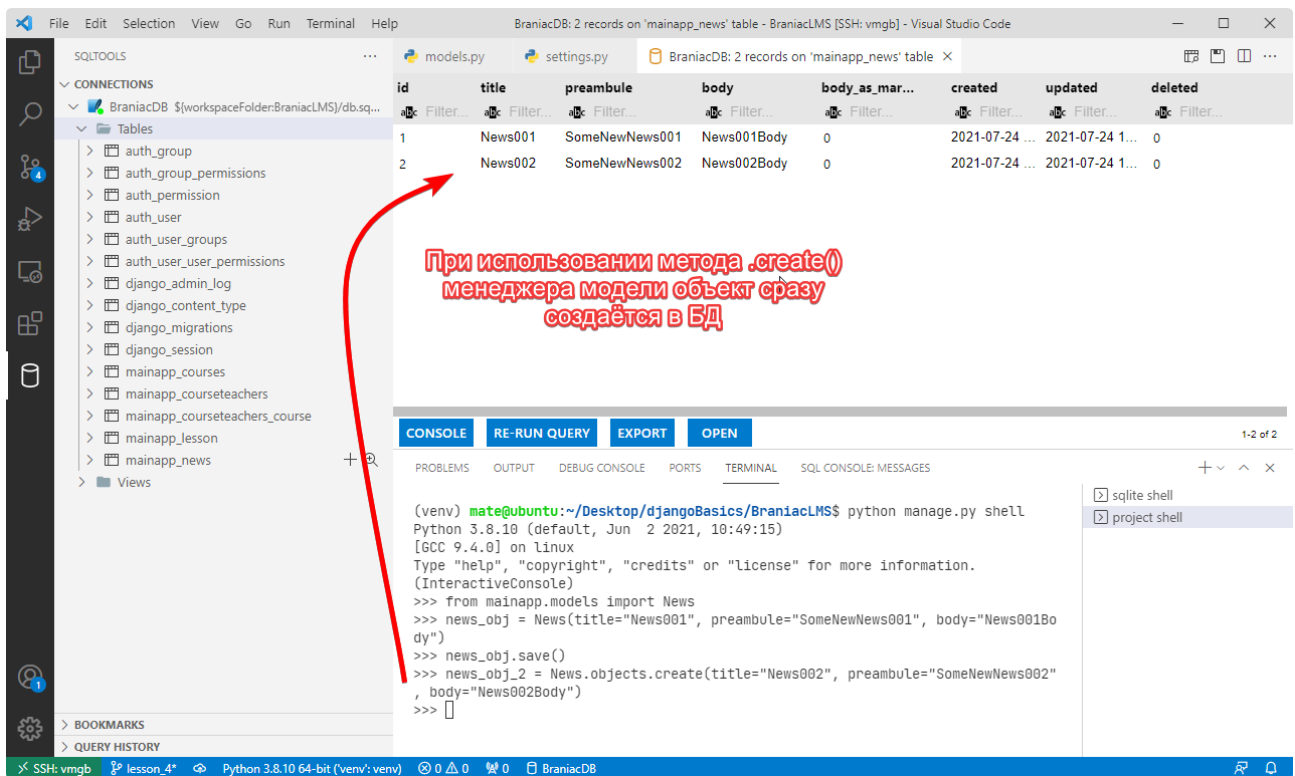


Рис. 15. Создание объекта через менеджера модели

Менеджер модели — это интерфейс, через который осуществляются запросы к БД. Стандартное расположение менеджера модели: в атрибуте `objects`. Весь набор методов менеджера модели есть на соответствующей [странице документации](#).

Для создания объекта в базе данных через менеджера модели надо ввести команду:

```

1 # Создание объекта с помощью менеджера модели
2 news_obj_2 = News.objects.create(
3     title="News002",
4     preamble="SomeNewNews002",
5     body="News002Body"
6 )

```

```
>>> news_obj.pk
1
>>> news_obj.id
1
>>> news_obj_2.pk
2
>>> news_obj_2.id
2
■
```

Рис. 16. Обращение к свойствам объектов модели

Метод возвращает созданный объект. В обоих случаях после сохранения объекта в базе появляется возможность обратиться к другим его свойствам, например, к id или pk.

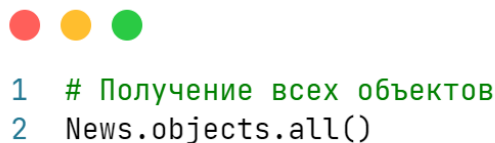
Получение объектов

Большинство операций получения объектов выполняется через менеджера моделей. Типичные операции получения объектов:

1. Получить все объекты.
2. Получить объекты, отфильтрованные по какому-либо параметру.
3. Получить объекты, исключая некоторые по какому-либо параметру.
4. Получить список значений из поля таблицы.
5. Получить конкретный объект.

Выборка всех объектов

Чтобы получить все объекты модели новостей для учебного проекта из БД, выполняется команда:



```
1 # Получение всех объектов
2 News.objects.all()
```

**Выборка всех объектов - это
выборка всех записей из БД
конвертированных в объекты
модели**

```
>>> News.objects.all()
<QuerySet [<News: 1 News001>, <News: 2 News002>]>
```

=

```
sqlite> select * from mainapp_news;
id      title      preamble      body      body_as_markdown      created      updated      deleted
-----
1      News001      SomeNewNews001      News001Body      0      2021-07-24 16:12:25.881826      2021-07-24 16:12:25.881889      0
2      News002      SomeNewNews002      News002Body      0      2021-07-24 16:18:33.034445      2021-07-24 16:18:33.034512      0
sqlite> ■
```

Рис. 17. Выборка данных, с точки зрения Django ORM и SQL-запроса

При выполнении запроса на получение списка объектов возвращается **неизменяемый** объект QuerySet. Он похож на обычный Python-список, но к нему применяются методы менеджера модели для последующего уточнения выборки.

На странице [официальной документации](#) все методы из списка Methods that return new QuerySets возвращают объект QuerySet.

Фильтрация

Для уточнённой выборки (фильтрации) используется метод менеджера модели `.filter()`. В качестве параметров для фильтрации в метод `.filter()` передаются свойства класса модели со значением, по которому отфильтровываются объекты.

Django ORM

```
>>> news_list = News.objects.filter(title="News001")
>>> news_list
<QuerySet [News: 1 News001]>
>>> print(news_list.query)
SELECT "mainapp_news"."id", "mainapp_news"."title", "mainapp_news"."preamble", "mainapp_news"."body", "mainapp_news"."body_as_markdown", "mainapp_news"."created", "mainapp_news"."updated", "mainapp_news"."deleted" FROM "mainapp_news" WHERE "mainapp_news"."title" = News001
>>> []
```

SQL

```
sqlite> SELECT "mainapp_news"."id", "mainapp_news"."title", "mainapp_news"."preamble", "mainapp_news"."body", "mainapp_news"."body_as_markdown", "mainapp_news"."created", "mainapp_news"."updated", "mainapp_news"."deleted" FROM "mainapp_news" WHERE "mainapp_news"."title" = "News001";
id      title      preamble      body      body_as_markdown      created      updated      deleted
-----
1       News001    SomeNewNews001  News001Body  0                2021-07-24 16:12:25.881826  2021-07-24 16:12:25.881889  0
sqlite> []
```

Рис. 18. Получение отфильтрованного списка объектов, с точки зрения Django ORM и SQL

Например, для получения объектов всех новостей, заголовков (title) которых равен значению News001, выполняется команда:



```
1 # Получение отфильтрованного списка
2 News.objects.filter(title="News001")
```

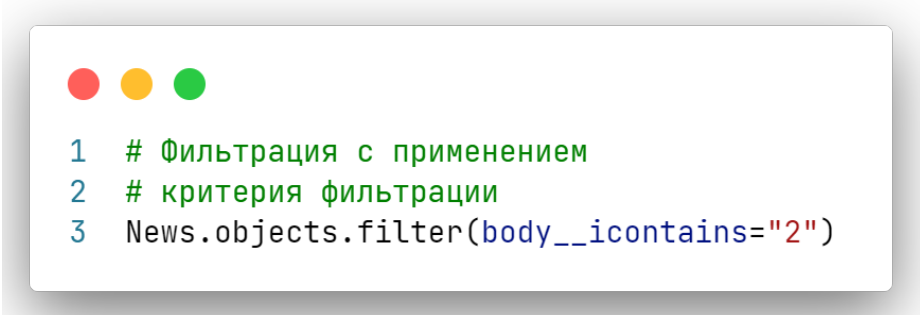
Если сохранить эту выборку в переменную (например, `news_list`), то sql-запрос распечатывается с применением команды:



```
1 # Вывод SQL-запроса
2 print(news_list.query)
```

Когда точное значение для фильтрации неизвестно, используется *критерий фильтрации* (модификатор фильтрации). Критерий фильтрации — это суффикс параметра фильтрации, указанный через двойное подчёркивание.

Например, если надо найти все новости, в теле (body) которых есть цифра 2, выполняется такой запрос:



```
1 # Фильтрация с применением
2 # критерия фильтрации
3 News.objects.filter(body__icontains="2")
```

```
>>> news_list = News.objects.filter(body__icontains="2")
>>> news_list
<QuerySet [<News: 2 News002>]>
>>> print(news_list.query)
SELECT "mainapp_news"."id", "mainapp_news"."title", "mainapp_news"."preamble",
"mainapp_news"."body", "mainapp_news"."body_as_markdown", "mainapp_news"."created", "mainapp_news"."updated", "mainapp_news"."deleted" FROM "mainapp_news" WHERE
"mainapp_news"."body" LIKE %2% ESCAPE '\'
```

Рис. 19. Запрос через Django ORM с критериями фильтрации

Критерий фильтрации `__icontains` говорит, что требуется выполнить поиск подстроки со значением 2 в строке значения поля `body` без учёта регистра.

Выборка в Django ORM «ленивая». Это значит, что реальный запрос в базу будет уходить только при обращении к самой выборке. Возвращаемый объект фильтрации `QuerySet` дополнительно фильтруется по необходимым условиям. Таким образом, в рамках одной выборки комбинируется последовательность фильтров. Например: `New.objects.filter(...).filter(...)`.

Все доступные критерии фильтрации описаны на [официальной странице документации](#) в разделе `Field lookups`.

Исключение объектов

Чтобы исключить объекты из выборки, надо:

1. Получить все объекты, исключая те, для которых верно значение параметра, если выборка новая.

2. Среди найденных объектов исключить те, для которых верно значение параметра, если проведена дополнительная фильтрация через объект `QuerySet`.

```
>>> news_list = News.objects.exclude(body__icontains="2")
>>> news_list
<QuerySet [News: 1 News001]>
>>> print(news_list.query)
SELECT "mainapp_news"."id", "mainapp_news"."title", "mainapp_news"."preamble",
"mainapp_news"."body", "mainapp_news"."body_as_markdown", "mainapp_news"."created", "mainapp_news"."updated", "mainapp_news"."deleted" FROM "mainapp_news" WHERE
NOT ("mainapp_news"."body" LIKE %2% ESCAPE '\') AND "mainapp_news"."body" IS NOT
NULL)
>>> █
```

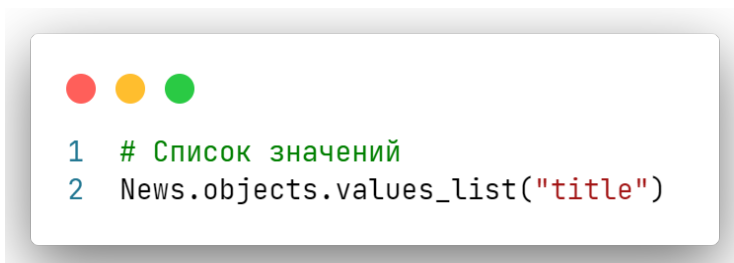
Рис. 20. Фильтрация исключением и итоговый выполняемый SQL-запрос

Исключение производится путём применения метода менеджера модели `.exclude()`. Параметры, которые передаются в качестве аргументов, аналогичны параметрам метода `.filter()`.

Получение списка значений

Зачастую требуется получить только некоторые значения из таблицы. Для этого применяется метод менеджера модели `.values_list()`.

Например, если надо получить все заголовки новостной ленты, выполняется такая выборка:



```
1 # Список значений
2 News.objects.values_list("title")
```

```
>>> news_list = News.objects.values_list("title")
>>> news_list
<QuerySet [News001], (News002),]>
>>> news_list = News.objects.values_list("title", flat=True)
>>> news_list
<QuerySet ['News001', 'News002']>
>>> print(news_list.query)
SELECT "mainapp_news"."title" FROM "mainapp_news"
>>> █
```

Рис. 21. Получение списка значений

Метод `.values_list()` принимает в качестве позиционных параметров имена полей, у которых требуются значения. Без указания дополнительных параметров он представит список кортежей, где значения распределяются согласно позициям переданных аргументов. Если вы запрашиваете значение одного

поля, то дополнительно указывается именованный параметр `flat=True`. В результате вернется список значений. Чтобы дальше работать с этим списком как с обычным, Python потребует дополнительно преобразовать, передав полученную выборку во встроенную функцию `list()` (объект `QuerySet` неизменяем).

Выборка конкретных объектов

Чтобы получить конкретные объекты, надо воспользоваться методом менеджера модели `.get()` и указать значение, по которому требуется произвести выборку.

Так, чтобы получить объект с `id`, равным 2, выполняется запрос:



Рис. 22. Получение конкретного объекта модели

```
>>> News.objects.get(pk=3)
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "/home/mate/Desktop/djangoBasics/venv/lib/python3.8/site-packages/django/db/models/manager.py", line 85, in manager_method
    return getattr(self.get_queryset(), name)(*args, **kwargs)
  File "/home/mate/Desktop/djangoBasics/venv/lib/python3.8/site-packages/django/db/models/query.py", line 435, in get
    raise self.model.DoesNotExist(
mainapp.models.News.DoesNotExist: News matching query does not exist.
```

Рис. 23. Ошибка получения несуществующего объекта

Метод `.get()` активирует исключение `DoesNotExist`, если подходящего объекта в таблице не найдено. Когда происходит выборка объекта с использованием метода `.get()`, надо обернуть этот участок кода в блок `try/except`.

Обновление данных в объектах

Чтобы обновить данные в БД, применяется два подхода:

1. Получение конкретного объекта. Установка нового значения для требуемого свойства. Сохранение объекта.

2. Использование метода менеджера модели.

Обновление через свойство полученного объекта

Так выглядит обновление объекта через установку нового значения в свойстве:



```
1 news_obj = News.objects.get(pk=1)
2 news_obj.title
3 # 'News001'
4 news_obj.title = "NewNews001"
5 news_obj.save()
```

models.py

settings.py

BraniacDB: 2 records on 'mainapp_news' table

id	title	preamble	body
abc Filter...	abc Filter...	abc Filter...	abc Filter...
1	NewNews001	SomeNewNews001	News001Body
2	News002	SomeNewNews002	News002Body

Рис. 24. Обновлённый объект в таблице БД

Обновление объекта через метод менеджера модели

У стандартного менеджера модели есть метод `.update_or_create()`. Он позволяет написать всего одну строку кода, но выполнить сразу несколько действий:

1. Проверить, есть ли объект в базе.
2. Если объект найден, установить заданное значение.
3. Если объекта нет, создать его с выставленными значениями.

Например, для обновления лида новости надо выполнить:



```
1 news_obj, created = News.objects.update_or_create(  
2     title="News002",  
3     defaults={"preamble":"NewPreambleNews002"}  
4 )
```

```
>>> news_obj, created = News.objects.update_or_create(title="News002", defaults=  
{"preamble":"NewPreambleNews002"})  
>>> news_obj  
<News: 2 News002>  
>>> created  
False  
>>> █
```

Рис. 25. Обновление объекта через метод менеджера модели

models.py

settings.py

BraniacDB: 2 records on 'mainapp_news' table X

id	title	preamble	body
abc Filter...	abc Filter...	abc Filter...	abc Filter...
1	NewNews001	SomeNewNews001	News001Body
2	News002	NewPreambleNews002	News002Body

Рис. 26. Обновлённый объект в таблице БД

Результат выполнения этого — кортеж, который распаковывается в переменные. Второй объект — указание на признак создания объекта — ответ на вопрос: «Объект сформировался?».

Удаление объектов

Удаление объектов из БД осуществляется методом `delete()`, который вызывается у объектов модели и выборки объектов.

Например, можно удалить объекты новостей, в теле которых содержится слово `2Body`:



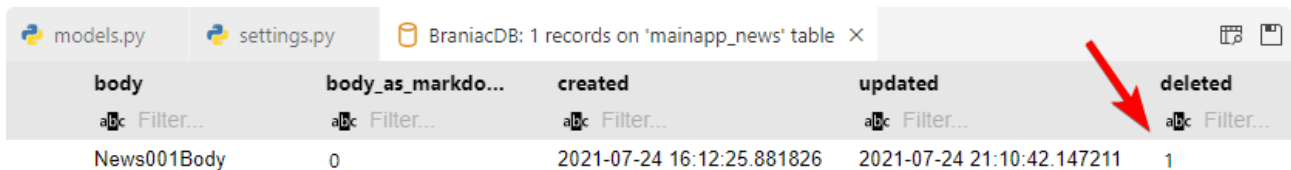
```
1 News.objects.filter(body__icontains="2Body").delete()
```



```
>>> News.objects.filter(body__icontains="2Body").delete()
(1, {'mainapp.News': 1})
>>> News.objects.first().delete()
>>> █
```

Рис. 27. Удаление объектов через вызов метода у выборки объектов и у конкретного объекта

В результате вернётся кортеж из значений. Первым будет значение общего количества удалённых объектов, вторым — словарь, где ключами обозначаются модели, из которых произошло удаление, а значениями — количество удалённых объектов.



body	body_as_markdo...	created	updated	deleted
News001Body	0	2021-07-24 16:12:25.881826	2021-07-24 21:10:42.147211	1

Рис. 28. Объект с выставленным признаком, что он удалён, с точки зрения таблицы в БД

В нашем учебном проекте вызов на выборке удалит записи из таблицы, а в случае вызова метода на объекте — выставит признак того, что объект удален.

Заполнение базы данными

Сейчас база учебного проекта практически пуста, но данные на страницах уже есть. Надо перенести их в базу и реализовать механизм вывода данных для пользователя, чтобы отображение стало динамичным.

В Django есть два способа занести данные в базу:

1. Фикстуры.
2. Миграции данных.

Фикстуры

Фикстуры — это файлы структурированных данных форматов JSON, XML, YAML. Они содержат данные для размещения в таблицах БД.

Для размещения фикстур используется путь <имя_приложения>\fixtures.

Основная цель использования фикстур: загрузка данных при запуске автоматизированных тестов.

Создание фикстуры

Фикстуры считаются обычными текстовыми файлами и позволяют загружать данные в базу. Команда `dumpdata` позволяет выгружать данные из базы в фикстуры. Данные по умолчанию выгружаются в

консоль, используя формат JSON. То есть чтобы выгрузить данные, надо перенаправить поток вывода в соответствующий файл.

В нашем учебном проекте данные содержатся в таблице новостей. Чтобы их выгрузить, надо указать модель в формате <имя_приложения>.<имя_модели>.

Вид команды выгрузки данных:

```
1 # Выгрузка данных в формате JSON
2 python manage.py dumpdata mainapp.News > mainapp/fixtures/001_news.json
```

The screenshot shows the Visual Studio Code interface with a database table view and a JSON export. The table view at the top shows a single record with the following data:

id	title	preamble	body	body_as_markdo...	created	updated	deleted
1	NewNews001	SomeNewNews001	News001Body	0	2021-07-24 16:12:25.881826	2021-07-24 21:10:42.147211	1

Below the table, the JSON export is shown in a text editor. The JSON data is as follows:

```
{
  "model": "mainapp.news",
  "pk": 1,
  "fields": {
    "title": "NewNews001",
    "preamble": "SomeNewNews001",
    "body": "News001Body",
    "body_as_markdown": false,
    "created": "2021-07-24T16:12:25.881Z",
    "updated": "2021-07-24T21:10:42.147Z",
    "deleted": true
  }
}
```

At the bottom, the terminal shows the command used to export the data:

```
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py dumpdata mainapp.News > mainapp/fixtures/001_news.json
```

Рис. 29. Выгрузка данных из базы формата JSON в фикстуру

Данные выгружаются в не отформатированном виде, кроме формата YAML, поэтому иногда приходится их форматировать дополнительно. Файлы фикстуры можно изменять под свои нужды.

После того как мы откорректировали файл фикстуры для таблицы новостей, данные выглядят так:

```
mainapp/fixtures/001_news.json

[
  {
    "model": "mainapp.news",
```

```

    "pk": 1,
    "fields": {
        "title": "Запуск нового курса по Python",
        "preamble": "Мы рады вам сообщить о запуске нового курса по Python для начинающих!",
        "body": "Python предназначен для разработки и изучения новых приложений. Мы предлагаем не просто изучить новый язык программирования, но и научиться применять его на практике. Этот курс полезен не только тем, кто хочет научиться программировать, но и тем, кому не хватает времени на выполнение практических заданий. Ещё это полезно тем, у кого нет возможности посещать курсы. Курс состоит из 7 встреч — одно занятие в неделю длительностью 3 часа.\r\nВстречи проводятся по воскресеньям с 14:00.",
        "body_as_markdown": false,
        "created": "2021-07-07T18:32:47.331Z",
        "updated": "2021-07-07T18:32:47.331Z"
    }
},
{
    "model": "mainapp.news",
    "pk": 2,
    "fields": {
        "title": "Урок по PHP в среду не состоится",
        "preamble": "Всем, кто не успел купить курс в понедельник, настоятельно рекомендую это сделать по ссылке в конце публикации.",
        "body": "Сегодня в Саратове не будет проходить открытый урок по PHP, который должен был состояться в среду на факультете компьютерных наук СГУ. Преподавательница, за которой закреплена аудитория, сказала, что заболела и не придёт. На следующий урок, запланированный на 28 февраля, перенесли всё, кроме курса «Веб-программирование». Об этом сообщили в группе факультета в социальной сети «ВКонтакте». Курс «Веб-программист» состоит из двух частей. Первая — общий курс, проходящий в течение 72 часов.",
        "body_as_markdown": false,
        "created": "2021-07-06T18:32:47.331Z",
        "updated": "2021-07-06T18:32:47.331Z"
    }
},
{
    "model": "mainapp.news",
    "pk": 3,
    "fields": {
        "title": "Всем руководителям подразделений подключится к Zoom",
        "preamble": "Все сотрудники подключены к Zoom, поэтому мы видим, что они делают и говорят.",

```

```

    "body": "Каждый из 5 руководителей будет в одном из своих офисов и использовать это приложение. Мы хотим, чтобы они могли видеть всё, что происходит на сайте, и просматривать предоставляемые отчёты. После того как сотрудники подключат ZOOM, важно, чтобы каждый секретарь, менеджер и прочие подключились ко всем 5 компьютерам.",
    "body_as_markdown": false,
    "created": "2021-07-05T18:32:47.331Z",
    "updated": "2021-07-05T18:32:47.331Z"
  }
},
{
  "model": "mainapp.news",
  "pk": 4,
  "fields": {
    "title": "Сегодня студенты всего мира отмечают праздник",
    "preamble": "Подробности — внутри...",
    "body": "Сегодня студенты всего мира отмечают праздник, который официально признаётся международным, но более распространён в таких странах, как Китай или Япония. Он называется Фестиваль студенческого пирога (Holly Fest) и празднуется каждый год в начале сентября. Этот праздник объединяет студентов во всём мире. Студенты отмечают начало нового учебного года и дарят друг другу подарки. Во многих азиатских странах этот день также знаменует начало нового года по китайскому календарю. День рождения — 9 июля, 23 года",
    "body_as_markdown": false,
    "created": "2021-07-04T18:32:47.331Z",
    "updated": "2021-07-04T18:32:47.331Z"
  }
},
{
  "model": "mainapp.news",
  "pk": 5,
  "fields": {
    "title": "Встречайте нового преподавателя направления DevOps",
    "preamble": "Дмитрий Шишмарёв работает в IT-бизнесе с 2001 года.",
    "body": "До прихода в компанию «1С-Битрикс» начинал карьеру в качестве системного администратора. В 2009 перешёл на должность DevOps-инженера, где и трудится по сей день, развиваясь в своей сфере. Дмитрий — сертифицированный специалист компании Bitrix. На его счету более 30 успешно реализованных проектов, среди которых разработка корпоративной системы управления веб-проектами на базе 1С-Bitrix",
    "body_as_markdown": false,
    "created": "2021-07-03T18:32:47.331Z",
    "updated": "2021-07-03T18:32:47.331Z"
  }
}

```

```

    }
  },
  {
    "model": "mainapp.news",
    "pk": 6,
    "fields": {
      "title": "JavaScript снова возглавил рейтинг самых отвратительных языков",
      "preamble": "И опять, и снова.",
      "body": "Рейтинг самых отвратительных языков программирования,
составленный британским изданием, возглавляет JavaScript. В опросе приняли
участие более 1000 специалистов по разработке ПО, работающих в различных
компаниях. При составлении списка учитывались такие свойства языка, как
сложность и простота обучения, а также производительность. Специалисты оценивали
язык по 10-балльной шкале, где один балл означал «ужасающий», а 10 баллов —
«отвратительный».",
      "body_as_markdown": false,
      "created": "2021-07-02T18:32:47.331Z",
      "updated": "2021-07-02T18:32:47.331Z"
    }
  }
]

```

При редактировании фикстуры надо соблюдать формат, в котором она создана, и для всех записей указывать параметр pk. Если этого не сделать, то при последующих загрузках фикстуры данные станут дублироваться: количество дублей будет равно количеству загрузок данных в базу.

Поскольку загрузка фикстуры, скорее всего, станет выполняться вручную, рекомендуется именовать файлы с числовым префиксом. Данные в таблицах часто ссылаются друг на друга, поэтому требуется соблюдать порядок загрузки. А числовой префикс не даст вам запутаться.

Фикстуры с данными для таблиц списка курсов, уроков и преподавателей приводятся в [приложении Б](#).

Загрузка фикстуры

Для загрузки данных используется команда `loaddata`. Аргументом к ней служит:

1. Имя файла фикстуры без расширения.
2. Путь (относительный или абсолютный) к файлу фикстуры.

Для загрузки данных из фикстуры новостей требуется выполнить команду:

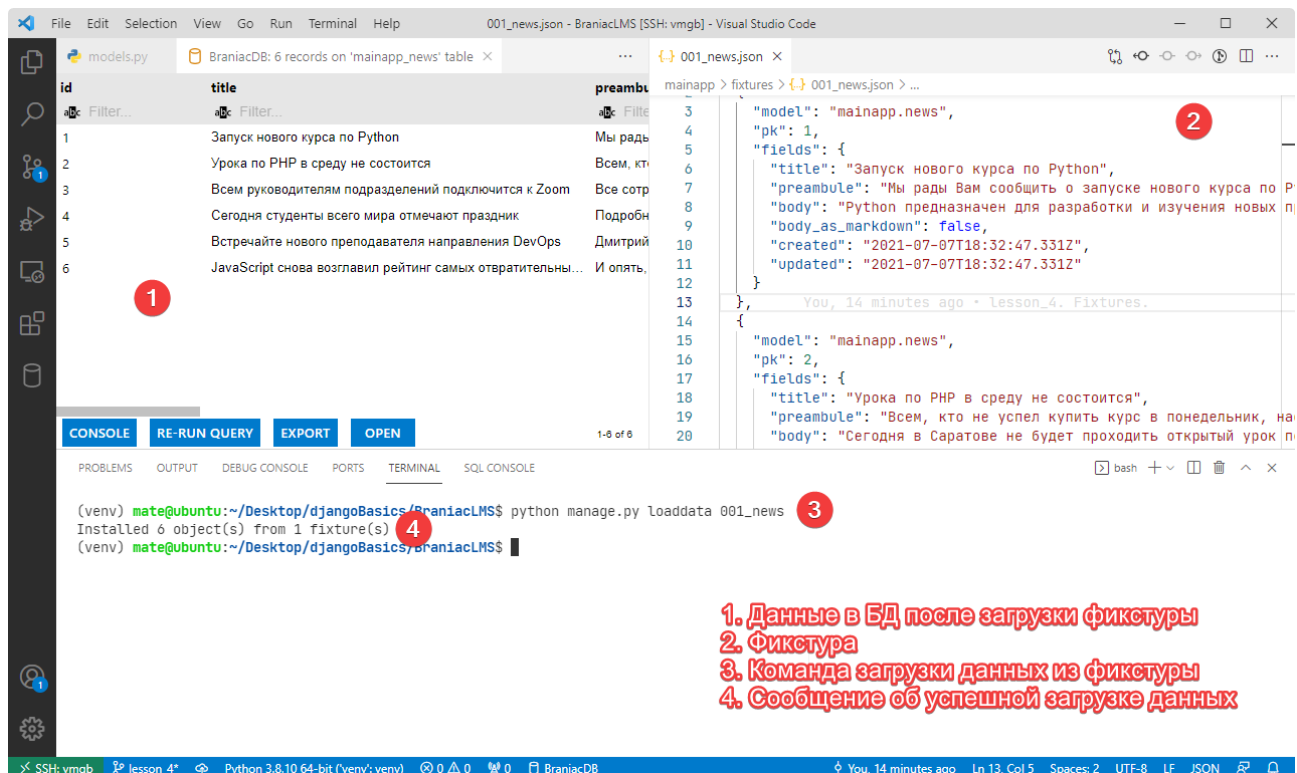
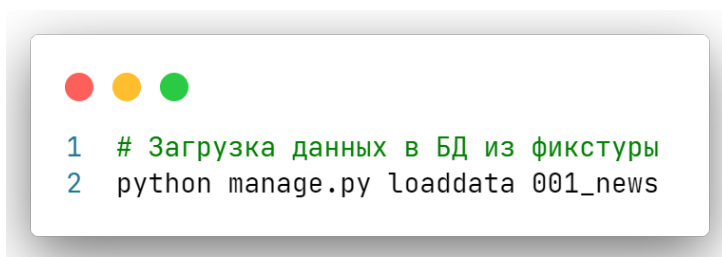


Рис. 30. Загрузка данных в базу из фикстуры

Если в разных приложениях есть фикстуры с одинаковым именем, а в команде загрузки указывается только имя файла фикстуры, то загрузка будет происходить из первого найденного файла. Порядок совпадает с порядком указания приложений в переменной `INSTALLED_APPS` в конфигурационном файле проекта.

```

(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py loaddata ./mainapp/fixtures/002_courses.json
Installed 8 object(s) from 1 fixture(s)
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py loaddata ./mainapp/fixtures/003_lessons.json
Installed 64 object(s) from 1 fixture(s)
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py loaddata ./mainapp/fixtures/004_teachers.json
Installed 7 object(s) from 1 fixture(s)
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$

```

Рис. 31. Загрузка данных из фикстур с указанием относительного пути к файлу

Преимущества и недостатки фикстур

Преимущества фикстур:

1. Удобство чтения и редактирования.
2. Множество поддерживаемых форматов.
3. Возможность автоматической генерации встроенными средствами для переноса данных на другие развёрнутые системы.
4. Возможность генерации фикстур сторонними инструментами. Это используется, например, для миграций проектов.
5. Возможность использования во время автотестов.

Недостатки фикстур:

1. Односторонний механизм (только внести данные).
2. Возможные ошибки из-за человеческого фактора.
3. Медленная работа.
4. Необходимость дополнительных библиотек (YAML-формат).

Миграции данных

Миграции показывают, как изменится БД. Внесение данных в БД также считается изменением. Этот механизм называется *миграцией данных*. Одно из явных преимуществ над фикстурами — возможность откатить изменения, если это предусмотрено разработчиком.

Миграции данных представляют собой обычный файл миграции, который генерируется командой создания миграций с ключом `--empty`. Затем в созданном файле прописываются требуемые изменения через Django ORM. Изменения, которые вносятся с применением миграции, включают не только добавление данных, но и на любые другие доступные операции: обновление, перемещение, удаление, изменение данных и т. д.

Для создания миграции выполняется команда:



```
1 # Создание пустой миграции
2 python manage.py makemigrations --empty --name data_migration mainapp
```

Команда читается так: создать пустой файл миграции с именем `data_migration` в приложении `mainapp`.

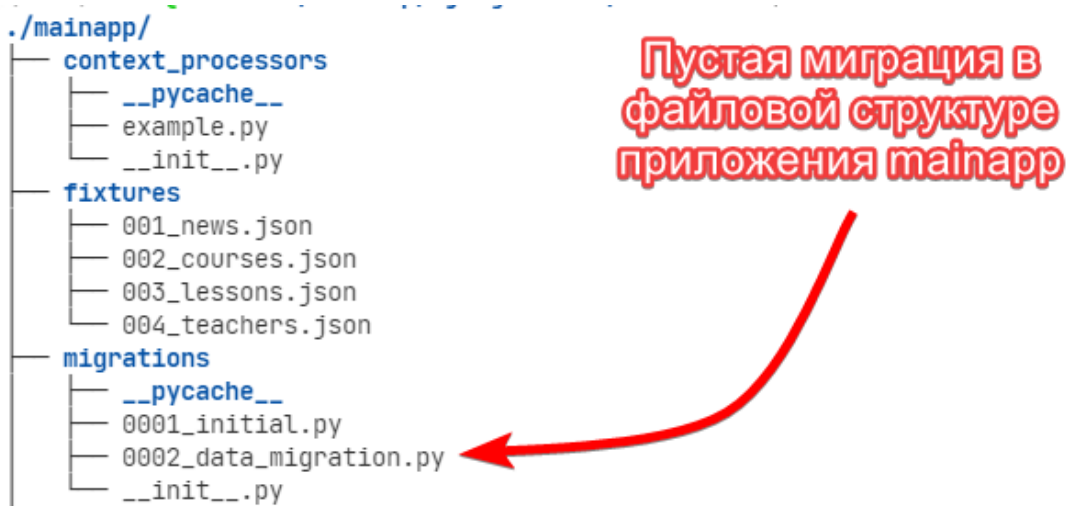


Рис. 32. Файл миграции данных в файловой структуре проекта

Вот миграция данных для заполнения таблицы новостей (полное содержимое файла):

```
mainapp/migrations/0002_data_migration.py

from django.db import migrations

def forwards_func(apps, schema_editor):
    # Get model
    News = apps.get_model("mainapp", "News")
    # Create model's objects
    News.objects.create(
        title="Запуск нового курса по Python",
        preamble="Мы рады вам сообщить о запуске нового курса по Python для \
начинающих!",
        body="Python предназначен для разработки и изучения новых приложений. \
Мы предлагаем не просто изучить новый язык программирования, но и \
научиться применять его на практике. Этот курс полезен не \
только тем, кто хочет научиться программировать, но и всем тем, \
кому не хватает времени на выполнение практических заданий, а также \
\
тем, у кого нет возможности посещать курсы. Курс состоит из 7 \
встреч — одно занятие в неделю длительностью 3 часа.\r\nВстречи \
проводятся по воскресеньям с 14:00.",
    )
    News.objects.create(
        title="Урока по PHP в среду не состоится",
        preamble="Всем, кто не успел купить курс в понедельник, настоятельно \
рекомендую это сделать по ссылке в конце публикации.",
```



```

        body="Сегодня в Саратове не будет проходить открытый урок по РНР, \
        который должен был состояться в среду на факультете компьютерных \
        наук СГУ. Преподавательница, за которой закреплена аудитория, \
        сказала что заболела и не придёт. На следующий урок, \
        запланированный на 28 февраля, перенесли всё, кроме курса \
        «Веб-программирование». Об этом сообщили в группе факультета в \
        социальной сети «ВКонтакте». Курс «Веб-программист» состоит из \
        двух частей. Первая — общий курс, идущий в течение 72 часов.",
    )
    News.objects.create(
        title="Всем руководителям подразделений подключится к Zoom",
        preamble="Все сотрудники подключены к Zoom, поэтому мы видим, что \
        они делают и говорят.",
        body="Каждый из 5 руководителей будет в одном из своих офисов с \
        использованием этого приложения. Мы хотим, чтобы они могли видеть \
        всё, \
        что происходит на сайте, просматривать отчёты, которые им \
        предоставляются. После того как сотрудники подключили ZOOM, нам \
        важно, чтобы каждый секретарь, менеджер и прочие были \
        подключены ко всем 5 компьютерам.",
    )
    News.objects.create(
        title="Сегодня студенты всего мира отмечают праздник",
        preamble="Подробности — внутри...",
        body="Сегодня студенты всего мира отмечают праздник, который \
        официально признан международным, но более распространён в \
        Таких азиатских странах, как Китай или Япония. Он называется \
        Фестиваль студенческого пирога (Holly Fest) и празднуется каждый \
        год в начале сентября. Этот праздник объединяет студентов во \
        всём мире. Они отмечают начало нового учебного года и дарят \
        друг другу подарки. Во многих азиатских странах этот день также \
        знаменует начало нового года по китайскому календарю. День \
        Рождения: 9 июля, 23 года",
    )
    News.objects.create(
        title="Встречайте нового преподавателя направления DevOps",
        preamble="Дмитрий Шишмарёв работает в IT-бизнесе с 2001 года.",
        body="До прихода в компанию «1С-Битрикс» начинал карьеру в качестве \
        системного администратора. В 2009 перешёл на должность \
        DevOps-инженера, где и трудится по сей день, развиваясь в своей \
        сфере. Дмитрий — сертифицированный специалист компании \
        Bitrix. На его счету более 30 успешно реализованных проектов, \
        среди которых: разработка корпоративной системы управления \

```

```

        веб-проектами на базе 1С-Bitrix",
    )
    News.objects.create(
        title="JavaScript снова возглавил рейтинг самых отвратительных \
        языков",
        preamble="И опять, и снова.",
        body="Рейтинг самых отвратительных языков программирования, \
        составленный британским изданием, возглавляет JavaScript. В опросе \
        приняли участие более 1000 специалистов по разработке ПО, \
        работающих в различных компаниях. При составлении списка \
        учитывались такие свойства языка, как сложность и простота \
        обучения, а также производительность. Специалисты оценивали язык \
        по 10-балльной шкале, где один балл означал «ужасающий», а 10 \
        баллов — «отвратительный».",
    )

def reverse_func(apps, schema_editor):
    # Get model
    News = apps.get_model("mainapp", "News")
    # Delete objects
    News.objects.all().delete()

class Migration(migrations.Migration):

    dependencies = [
        ("mainapp", "0001_initial"),
    ]

    operations = [
        migrations.RunPython(forwards_func, reverse_func),
    ]

```

В этом коде выделяется несколько особенностей:

1. Основной операцией в миграции станет запуск функций Python через отдельную функцию модуля миграций: `migrations.RunPython(forwards_func, reverse_func)`. Запущенные функции будут переданы как объекты (без скобок).

2. Функций для запуска две: `forwards_func`, `reverse_func`. Первая служит для применения миграции, вторая — для её отката.
3. Получение модели происходит не как импорт соответствующего класса из модуля, а как результат вызова функции: `apps.get_model("mainapp", "News")`. Возвращаемый объект считается обычным классом модели.
4. В функции отката миграции происходит удаление данных, которые внесены в случае применения. Но это не единственно верный вариант: всё зависит от условий задачи.
5. В блоке `dependencies` указана зависимость одной миграции от других. Так система Django ORM понимает, в какой последовательности применять миграции всего проекта. Не рекомендуется изменять этот блок.

Названия функций можно изменять на своё усмотрение. Но обязательным указывается правильное количество входных параметров.

Откат миграций

Применение миграций — это процесс последовательного внесения изменений в базе данных в соответствии с кодом, указанным в файлах миграций. Соответственно, откат миграций отменяет изменения в БД. Чтобы откатить миграции, в команде миграции последним позиционным аргументом надо указать без расширения тот файл, до которого вы собираетесь произвести откат.

Например, в миграции данных указывается зависимость от миграции `0001_initial`. Значит, команда отмены будет выглядеть так:



```
1 # Откат миграции
2 python manage.py migrate mainapp 0001_initial
```

Команда читается так: произвести процедуру миграции для приложения `mainapp` до состояния на момент применения `0001_initial`.

```
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ python manage.py migrate mainapp 0001_initial
Operations to perform:
  Target specific migration: 0001_initial, from mainapp
Running migrations:
  Rendering model states... DONE
  Unapplying mainapp.0002_data_migration... OK
(venv) mate@ubuntu:~/Desktop/djangoBasics/BraniacLMS$ █
```

Рис. 33. Откат миграции

Решение типовых проблем учебного проекта

Часто в учебных проектах возникают проблемы, связанные с процедурой создания или применения миграций. В нашем случае проще всего разрешить их через этот алгоритм:

1. Удалить файл базы данных `db.sqlite3`.
2. Удалить файлы миграций из папки `migrations` (все, кроме `__init__.py`). Рекомендуется предварительно сделать резервную копию миграции данных.
3. Заново создать все миграции для всех приложений.
4. Произвести процедуру миграции и проверить статус в консоли — у всех должен он быть ОК.

Бинарные данные

Вопрос хранения бинарных данных — медиафайлов, файлов в бинарных форматах — считается одним из самых популярных среди новичков.

Лучше не хранить бинарные данные в базе. Проще сохранять в БД указание пути к ним. Например, в фикстуре по наполнению данными страницы курсов (`mainapp/fixtures/002_courses.json`) есть ссылка на файл с картинкой: `"cover": "img/no_image.svg"`.

Вывод данных из базы на страницы

После того как данные оказались в таблицах, их надо вывести на страницы пользователей. Все данные попадают сюда из контекста шаблонов. Значит, для вывода данных на страницы надо реализовать выборку данных при подготовке контекста.

Весь исходный код для изменившихся файлов — в [приложении В](#).

На страницу новостей требуется вывести данные, которыми управляет соответствующая модель. Код контроллера, отвечающий за страницу новостей, примет вид:

mainapp/views.py

```
class NewsPageView(TemplateView):
    template_name = "mainapp/news.html"

    def get_context_data(self, **kwargs):
        # Get all previous data
        context = super().get_context_data(**kwargs)
        # Create your own data
        context["news_qs"] = mainapp_models.News.objects.all()[:5]
        return context
```

При использовании выбранных контроллеров (TemplateView) в выборке учитывается количество объектов, которые отображаются на странице. Если объектов и запросов много, система зависнет. Для ограничения выборки по количеству используется синтаксис срезов списков. Таким образом, в контекст страницы новостей попадёт срез [:5] из пяти первых элементов выборки.

Цикл вывода объектов в шаблоне:

```
mainapp/templates/mainapp/news.html

{% for item in news_qs %}
<div class="card my-2">
  <div class="card-body">
    <h5 class="card-title">{{ item.title }}</h5>
    <h6 class="card-subtitle mb-2 text-muted">
      {{ item.created|date:"Y-m-d h-i-s" }}
    </h6>
    <p class="card-text">{{ item.preamble }}</p>
    <a href="{% url 'mainapp:news_detail' pk=item.pk %}"
      class="card-link">Подробнее</a>
  </div>
</div>
{% endfor %}
```

Теперь итерирование идёт по выборке из объектов модели, а все данные в шаблоне подставляются из атрибутов объектов.

Запуск нового курса по Python

2021-07-07 06-32-47

Мы рады Вам сообщить о запуске нового курса по Python для начинающих!

[Подробнее](#)

Запуск нового курса по Python

Python предназначен для разработки и изучения новых приложений. Мы предлагаем не просто изучить новый язык программирования, но и научиться применять его на практике. Данный курс будет полезен не только тем, кто хочет научиться программировать, но и всем тем, кому не хватает времени на выполнение домашних заданий, а также тем, у кого нет возможности посещать курсы. Курс состоит из 7 встреч (одно занятие в неделю длительностью 3 часа). Встречи проводятся по воскресеньям с 14:00.

[НАЗАД](#)

Рис. 34. Переход в детальное описание отображает содержимое новости

Чтобы отобразить новость полностью, надо добавить соответствующий контроллер:

mainapp/views.py

```
from django.shortcuts import get_object_or_404

class NewsPageDetailView(TemplateView):
    template_name = "mainapp/news_detail.html"

    def get_context_data(self, pk=None, **kwargs):
        context = super().get_context_data(pk=pk, **kwargs)
        context["news_object"] = get_object_or_404(mainapp_models.News, pk=pk)
        return context
```

Здесь стоит обратить внимание на функцию `get_object_or_404`. Она принимает на вход один позиционный параметр — модель и неограниченное количество именованных аргументов, по которым будет производиться выборка из этой модели. Если объект найден, он вернётся в качестве результата, наоборот — система выдаст ошибку 404 (объект не найден). Чтобы каждый раз не

обрабатывать ошибку, которая возникает при выборке объекта через метод менеджера модели `get`, используется эта обёртка.

Исходный код шаблона вывода детального отображения новости есть в соответствующем приложении. Для использования перехода в детальное отображение надо скорректировать диспетчер адресов приложения.

Один из атрибутов логического типа модели `Courses` — `description_as_markdown`. Он позволяет скорректировать вывод описания курса, если этот вывод дополнительно размечен в `markdown`. Для корректной работы поля требуется дополнительно установить расширение:



```
1 # Установка расширения markdownify
2 pip install django-markdownify
```

Требуется также подключить это расширение как приложение в конфигурационном файле:

config/settings.py

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "markdownify.apps.MarkdownifyConfig",
    "mainapp",
]
```

Подключим шаблонный фильтр для корректного отображения соответствующей разметки в шаблоне:

mainapp/templates/mainapp/courses_detail.html

```
{% load static markdownify %}

<div class="col-12">
    {% if course_object.description_as_markdown %}
    {{ course_object.description|markdownify }}
    {% else %}
    {{ course_object.description }}
    {% endif %}
</div>
```

```
{% endif %}  
</div>
```

Автоматическая фильтрация удалённых объектов

Для многих моделей мы задействовали механизм, при котором объект не удаляется, а лишь помечается удалённым. Чтобы на этапе выборки автоматически отфильтровывать такие объекты, используется приём по изменению методов менеджера модели.

Общий алгоритм выбранного подхода:

1. Создать свой менеджер модели через наследование от стандартного.
2. Скорректировать метод получения выборки.
3. Зарегистрировать новый менеджер в модели.

В качестве примера рассмотрим модель курсов. Её новый менеджер будет выглядеть так:

mainapp/models.py

```
class CoursesManager(models.Manager):  
    def get_queryset(self):  
        return super().get_queryset().filter(deleted=False)
```

Модель курсов примет вид:

mainapp/models.py

```
class Courses(models.Model):  
    objects = CoursesManager()  
  
    name = models.CharField(max_length=256, verbose_name="Name")  
    description = models.TextField(  
        verbose_name="Description", blank=True, null=True  
    )  
    description_as_markdown = models.BooleanField(  
        verbose_name="As markdown", default=False  
    )  
    cost = models.DecimalField(  
        max_digits=8, decimal_places=2, verbose_name="Cost", default=0  
    )  
    cover = models.CharField(  
        max_length=25, default="no_image.svg", verbose_name="Cover"  
    )
```



```
created = models.DateTimeField(auto_now_add=True, verbose_name="Created")
updated = models.DateTimeField(auto_now=True, verbose_name="Edited")
deleted = models.BooleanField(default=False)

def __str__(self) -> str:
    return f"{self.pk} {self.name}"

def delete(self, *args):
    self.deleted = True
    self.save()
```

Общепринятое расположение менеджера модели — в свойстве `objects`, поэтому новый менеджер также определится в нём.

Теперь при любой выборке сначала фильтруются все удалённые записи, а потом произойдёт выборка по оставшимся условиям.

Весь исходный код для этого раздела представлен в [приложении В](#).

Заключение

Почти все современные приложения задействуют механизмы по хранению данных в базе данных. Это удобно:

- можно легко фильтровать данные;
- быстро и точно получать требуемые данные;
- аккумулировать данные в виде комплексных таблиц и т. д.

Django предоставляет свой слой абстракции над базами данных (система ORM), что позволяет разработчику не смотреть на детали реализации конкретной СУБД, а сконцентрироваться на бизнес-логике. Система ORM покрывает большую часть задач, что потребуется при разработке проекта, а также несёт в себе накладные расходы в виде избыточных запросов и потере времени при конвертации «результат запроса — python-объекты». Несомненный плюс подобного подхода — увеличение скорости разработки и простота поддержки кодовой базы проекта.

Практическое задание

(*) — задания повышенной сложности, а (**) — самые сложные задания, поэтому их выполнять необязательно. Чтобы сдать практическое задание, сформируйте pull request на GitHub (описание в [инструкции](#)) и приложите ссылку на него в соответствующем разделе.

1. Создайте модели для новостей, курсов, уроков и преподавателей. Используйте различные типы полей:

- a. набор символов (CharField);
 - b. текстовое (TextField);
 - c. с логическим значением (BooleanField);
 - d. дата-время (DateTimeField);
 - e. со значением и плавающей точкой (DecimalField);
 - f. ссылочный тип (ForeignKey, ManyToManyField).
2. Сформируйте файлы миграции для созданных моделей. Через механизм миграций убедитесь в их применимости.
3. Заполните из консоли, подключённой к проекту, данными одну из моделей. Затем выполните создание фикстуры. Фикстуру наполните данными и проверьте корректность применения.
4. Создайте пустую миграцию и наполните её опциями для заполнения таблицы в БД и для возможности отката.
5. (*) Создайте отдельные миграции со своим уникальным именем для каждой модели.
6. (*) Создайте миграции данных для всех моделей.
7. (**) После создания всех моделей и наполнения их данными посредством механизма миграций перенесите данные из одного поля через промежуточную модель. Для каждой операции — отдельная миграция.
 - a) Перенесите данные в другую модель.
 - b) Очистите данные в исходной модели.
 - c) Перенесите данные в исходную модель.
 - d) Удалите промежуточную модель.

Глоссарий

Критерии фильтрации — суффикс к свойству модели, указывая который уточняется выборка.

Менеджер моделей — вспомогательный класс моделей, объект которого привязан к модели и позволяет осуществлять запросы в базу данных.

Миграции — термин, который используется в Django ORM в двух значениях:

- файл, содержащий изменения, вносимые в базу;
- механизм, благодаря которому изменения попадают в базу.

Миграции данных — файл, содержащий функции внесения данных в таблицы во время процедуры миграции.

Модели — описание таблиц баз данных в системе Django ORM.

Первичный ключ (primary key) — один из потенциальных ключей отношения в реляционной модели данных, выбранный в качестве основного ключа или ключа по умолчанию.

Принцип толстых моделей — принцип, согласно которому вся логика взаимодействия с объектами класса модели описывается в рамках класса этой модели. Позволяет соблюдать принцип DRY.

СУБД — система управления базой данных.

Фикстуры — текстовые файлы структурированных данных для внесения в таблицы через механизм загрузки или при использовании автоматизированных тестов.

Целевая страница (landing page), или «посадочная страница» — это веб-страница, основная задача которой — сбор контактов целевой аудитории. Используется для усиления эффективности рекламы и увеличения аудитории. Обычно лендинг содержит информацию о товаре или услуге.

ORM (Object-Relational Mapping), или объектно-реляционное отображение, преобразование — технология программирования, которая связывает базы данных с концепциями объектно-ориентированных языков программирования, создавая «виртуальную объектную базу данных».

SQL (structured query language), или «язык структурированных запросов» — декларативный язык программирования. Применяется для создания, модификации и управления данными в реляционной базе, управляемой соответствующей системой.

Дополнительные материалы

1. Документация по настройке подключения различных СУБД:
 - a. [Django and databases](#).
 - b. [Database settings](#).
2. Обсуждение использования [системы ORM на stackoverflow](#).
3. [Comparing Database Management Systems](#).
4. Документация по типам полей в Django ORM:
 - a. [Общие поля](#), характерные для всех используемых СУБД.
 - b. [Поля](#), характерные для PostgreSQL.
5. О markdown
 - a. [Краткое руководство](#).
 - b. [Wiki page](#).
6. [Магические методы в Python](#).
7. [Django model's API](#).
8. [Model Meta options](#).
9. Документация для выполнения SQL-запросов для некоторых СУБД:
 - a. [Command Line Shell For SQLite](#) и [SQLite Commands](#).
 - b. [PostgreSQL Documentation: The SQL Language](#).
 - c. [MariaDB](#) и [MySQL](#).
10. [Документация по миграциям данных](#):

- a. [Запуск функций Python во время миграций.](#)

Приложение А. Создание моделей

mainapp/models.py

```
from django.db import models

class News(models.Model):
    title = models.CharField(max_length=256, verbose_name="Title")
    preamble = models.CharField(max_length=1024, verbose_name="Preamble")
    body = models.TextField(blank=True, null=True, verbose_name="Body")
    body_as_markdown = models.BooleanField(
        default=False, verbose_name="As markdown"
    )
    created = models.DateTimeField(
        auto_now_add=True, verbose_name="Created", editable=False
    )
    updated = models.DateTimeField(
        auto_now=True, verbose_name="Edited", editable=False
    )
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return f"{self.pk} {self.title}"

    def delete(self, *args):
        self.deleted = True
        self.save()

class Courses(models.Model):
    name = models.CharField(max_length=256, verbose_name="Name")
    description = models.TextField(
        verbose_name="Description", blank=True, null=True
    )
    description_as_markdown = models.BooleanField(
        verbose_name="As markdown", default=False
    )
    cost = models.DecimalField(
        max_digits=8, decimal_places=2, verbose_name="Cost", default=0
    )
    cover = models.CharField(
```

```

        max_length=25, default="no_image.svg", verbose_name="Cover"
    )
    created = models.DateTimeField(auto_now_add=True, verbose_name="Created")
    updated = models.DateTimeField(auto_now=True, verbose_name="Edited")
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return f"{self.pk} {self.name}"

    def delete(self, *args):
        self.deleted = True
        self.save()

class Lesson(models.Model):
    course = models.ForeignKey(Courses, on_delete=models.CASCADE)
    num = models.PositiveIntegerField(verbose_name="Lesson number")
    title = models.CharField(max_length=256, verbose_name="Name")
    description = models.TextField(
        verbose_name="Description", blank=True, null=True
    )
    description_as_markdown = models.BooleanField(
        verbose_name="As markdown", default=False
    )
    created = models.DateTimeField(
        auto_now_add=True, verbose_name="Created", editable=False
    )
    updated = models.DateTimeField(
        auto_now=True, verbose_name="Edited", editable=False
    )
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return f"{self.course.name} | {self.num} | {self.title}"

    def delete(self, *args):
        self.deleted = True
        self.save()

class Meta:
    ordering = ("course", "num")

```

```

class CourseTeachers(models.Model):
    course = models.ManyToManyField(Courses)
    name_first = models.CharField(max_length=128, verbose_name="Name")
    name_second = models.CharField(max_length=128, verbose_name="Surname")
    day_birth = models.DateField(verbose_name="Birth date")
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return "{0:0>3} {1} {2}".format(
            self.pk, self.name_second, self.name_first
        )

    def delete(self, *args):
        self.deleted = True
        self.save()

```

[Вернуться к теме.](#)

Приложение Б. Фикстуры

Фикстура курсов

mainapp/fixtures/002_courses.json

```

[
  {
    "model": "mainapp.courses",
    "pk": 1,
    "fields": {
      "name": "Web Python",
      "description": "Курс Web Python позволит вам освоить язык программирования
Python, а также разобраться с его основными аспектами: модулями, парсерами и
регулярными выражениями. Курс направлен на то, чтобы вы научились составлять
эффективные программы для решения своих задач. Он научит вас создавать
собственные веб-приложения для работы с базами данных и сайтами. Вы сможете
использовать Python для изучения Java, Ruby, PHP, C, C ++, C # или любого
другого языка программирования.\r\nНа курсе Web Python проходит изучение
веб-фреймворков Django, Bootstrap и Flask.\r\nВо втором семестре вы изучите
такие инструменты и методы, как:\r\n\r\n* PHP и MySQL;\r\n* Работа с файлами
DOC, EXCEL, PATH;\r\n* Реляционная и объектная модель базы данных;\r\n* Создание
и управление пакетами;\r\n* Как работать с FTP-сервером;\r\n* Настройка среды и
создание сайтов;\r\n* Веб-серверы;\r\n* Конфигурационное управление в Unix и

```

```
Windows;\r\n* Инструменты для создания сайта;\r\n* Интеграция внешних
приложений;\r\n* Расширения;",
  "description_as_markdown": true,
  "cost": "10000.50",
  "cover": "img/course001.jpg",
  "created": "2021-07-11T08:24:53.084Z",
  "updated": "2021-07-11T09:06:35.322Z"
}
},
{
  "model": "mainapp.courses",
  "pk": 2,
  "fields": {
    "name": "Web Golang",
    "description": "На курсе Web Golang вы изучите основы Go, получите
практические навыки реализации проектов на Golang и научитесь работать в
распределённых средах разработки. Вы узнаете, что такое Go, как компилировать
код Go и писать тесты. Выясните, как работает веб-сервер. Курс ведёт Виктор
Иванов\r\nСертифицированный специалист Google Developer Expert (Certified
Technical Professional).\r\nАвтор учебника «Веб-разработка на Go». На курсе вы
изучите основы языка программирования Go и получите практический опыт реализации
проектов на Go.",
    "description_as_markdown": false,
    "cost": "9009.99",
    "cover": "img/course002.jpg",
    "created": "2021-07-11T08:28:07.397Z",
    "updated": "2021-07-11T09:06:40.127Z"
  }
},
{
  "model": "mainapp.courses",
  "pk": 3,
  "fields": {
    "name": "Web JavaScript",
    "description": "Курс JavaScript ведёт преподаватель Дмитрий Кокшаров. Он
преподаёт в разных вузах, в том числе в МФТИ. Дмитрий считает, что курс — это
база знаний, которая пригодится в жизни. \r\nКурс JavaScript состоит из модулей,
один из которых — «Основы функционального программирования». Этот модуль
рассчитан на тех, кто не знает основы JavaScript или только начал заниматься
этим языком. Вы узнаете, как устроены функции, какие бывают типы данных, как
применять модификаторы.\r\nВ программе JavaScript:\r\n\r\n* Основные паттерны
проектирования (design patterns) \r\n* Разработка на чистом JavaScript\r\n*
ReactJS и AngularJS\r\n* ООП на JavaScript \r\n* Основы HTML 5 и CSS3\r\n*
```

Javascript ES6 и ES7\r\n* Кроссбраузерные основы\r\n\r\nПогрузитесь в динамичный мир современной JS-разработки. Курс JavaScript полезен как новичкам, так и опытным разработчикам, желающим изучить новые технологии и получить новые знания.",

```
    "description_as_markdown": true,
    "cost": "2500.15",
    "cover": "img/course003.jpg",
    "created": "2021-07-11T08:31:20.301Z",
    "updated": "2021-07-11T09:06:46.902Z"
  }
},
```

```
{
  "model": "mainapp.courses",
  "pk": 4,
  "fields": {
    "name": "Web Java",
    "description": "На курсе Java Вы изучите язык и фреймворк Java с самых азов до продвинутого уровня.\r\nА главное – получите теоретические и практические навыки для создания, отладки и тестирования своих программ. В итоге вы сможете выполнить проекты любой степени сложности, используя различные среды разработки и инструменты: от Eclipse IDE до IntelliJ IDEA, не говоря уже о современных облачных решениях от Google и GitHub.\r\nКурс Java ведёт сильный преподаватель Дмитрий. Дмитрий – живой пример профессионального, увлечённого и целеустремлённого преподавателя. С первых занятий, уже на первом уроке вы поймёте, что преподаватель заинтересован в том, чтобы его ученики не просто получили теоретические знания, а приобрели практические навыки.",
```

```
    "description_as_markdown": false,
    "cost": "10000.00",
    "cover": "img/course004.jpg",
    "created": "2021-07-11T08:34:10.771Z",
    "updated": "2021-07-11T09:06:52.015Z"
  }
},
```

```
{
  "model": "mainapp.courses",
  "pk": 5,
  "fields": {
    "name": "Web PHP",
    "description": "На курсе PHP вы изучите основы PHP и получите навыки работы с ним: работа с базами данных, графикой, созданием сайтов, работа в PHPedit, обработка исключений, управление памятью, работа со строками и с файлами. Курс PHP позволяет получить навыки программирования на этом языке. После прохождения курса вы сможете создавать собственные веб-сайты с системой
```


управления, а также применять свои знания для создания полноценных информационных систем.\r\nКурс PHP ведёт разработчик из Сбербанка, который умеет делать сложные вещи из простых. Он расскажет, как сделать сайт, не имея ничего, кроме базовых знаний, и поделится важными секретами.

«У меня было достаточно времени, чтобы посмотреть на PHP с нуля и понять, что он крутой. Как-то раз меня спросили, почему я не пишу код сам, а только консультирую других. Я ответил: „Потому что мне нравится помогать людям“. Но для этого надо уметь всё.\r\nPHP — это сложная штука.»",

```
"description_as_markdown": false,
"cost": "9500.78",
"cover": "img/course005.jpg",
"created": "2021-07-11T08:36:21.087Z",
"updated": "2021-07-11T09:06:57.580Z"
```

```
}
```

```
},
```

```
{
```

```
"model": "mainapp.courses",
```

```
"pk": 6,
```

```
"fields": {
```

```
"name": "Python AI",
```

```
"description": "На курсе Python AI вы изучите создание простых нейросетей для задач машинного зрения.\r\nНаучитесь создавать и развивать более сложные нейросети.\r\nКурс рассчитан:\r\n\r\n* Для тех, кому интересен мир машинного обучения, создания ИИ и робототехники.\r\n\r\n* Для специалистов в области физики, математики, программирования на Python.\r\n\r\n* Для всех, кто хочет прокачать свои навыки в Python, а также понять алгоритм работы нейронных сетей и научиться применять их на практике.\r\n\r\nКурс Python AI ведёт преподаватель школы \"Фабрика Открытий\" Денис Мигачёв.\r\nОн изучает математику и IT, много программирует и делает игры. Денис расскажет вам про инструменты, которые пригодятся для создания ИИ, и даст практические задания — написать свою игру на Python."
```

```
"description_as_markdown": true,
"cost": "6660.66",
"cover": "img/course006.jpg",
"created": "2021-07-11T08:39:15.369Z",
"updated": "2021-07-11T09:07:04.253Z"
```

```
}
```

```
},
```

```
{
```

```
"model": "mainapp.courses",
```

```
"pk": 7,
```

```
"fields": {
```

```
"name": "DevOps",
```

```

    "description": "На курсе DevOps вы изучите основы автоматизации с Python,
Git, Docker и узнаете про GitHub, Jenkins и Kubernetes. Поймёте, как всё
работает и зачем это нужно.\r\nКурс состоит из 12 онлайн-семинаров,
длительностью 90 минут. Он подходит опытным программистам, которые хотят
усовершенствовать свои навыки в области управления проектами.\r\nНа курсе вы
изучите:\r\n\r\n* Основные возможности и функции Git\r\n* Как создавать,
добавлять и удалять ветки, коммиты и теги\r\n* Как назвать и использовать
ветку",
    "description_as_markdown": true,
    "cost": "7770.77",
    "cover": "img/course007.jpg",
    "created": "2021-07-11T08:40:33.347Z",
    "updated": "2021-07-11T09:07:11.184Z"
  }
},
{
  "model": "mainapp.courses",
  "pk": 8,
  "fields": {
    "name": "3D-моделирование",
    "description": "Курс 3D-моделирования ведёт практикующий архитектор с
высшим архитектурным образованием, со стажем преподавания более 7 лет.\r\nНа
курсе 3D-моделирования вы изучите программы:\r\n\r\n* 3D-Max\r\n* Blender\r\n*
Rhino\r\n* Maya\r\n* AutoCAD (V-Ray)\r\n\r\nНаучитесь:\r\n\r\n1. Создавать
модели для разработки приложений.\r\n2. Создавать дизайн мебели.\r\n3. Создавать
реалистичные текстуры и освещение.\r\n4. Создавать анимации.\r\n5. Настраивать
рабочий процесс в программах.\r\n6. Работать с 3D Studio Max, Blender,
Rhinoceros.\r\n7. Использовать инструменты программы AutoCAD.\r\n8. Создавать
собственные проекты или адаптировать модели 3D-программ под свои нужды.",
    "description_as_markdown": true,
    "cost": "5900.00",
    "cover": "img/no_image.svg",
    "created": "2021-07-11T08:58:28.830Z",
    "updated": "2021-07-11T09:07:16.651Z"
  }
}
]

```

Фикстура уроков

mainapp/fixtures/003_lessons.json

```
[
  {
    "model": "mainapp.lesson",
    "pk": 11,
    "fields": {
      "course": 1,
      "num": 1,
      "title": "Lesson 1",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 12,
    "fields": {
      "course": 1,
      "num": 2,
      "title": "Lesson 2",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 13,
    "fields": {
      "course": 1,
      "num": 3,
      "title": "Lesson 3",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 14,
```

```

    "fields": {
      "course": 1,
      "num": 4,
      "title": "Lesson 4",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 15,
    "fields": {
      "course": 1,
      "num": 5,
      "title": "Lesson 5",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 16,
    "fields": {
      "course": 1,
      "num": 6,
      "title": "Lesson 6",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 17,
    "fields": {
      "course": 1,
      "num": 7,
      "title": "Lesson 7",

```

```

        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 18,
    "fields": {
        "course": 1,
        "num": 8,
        "title": "Lesson 8",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 21,
    "fields": {
        "course": 2,
        "num": 1,
        "title": "Lesson 1",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 22,
    "fields": {
        "course": 2,
        "num": 2,
        "title": "Lesson 2",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
}

```

```

    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 23,
    "fields": {
      "course": 2,
      "num": 3,
      "title": "Lesson 3",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 24,
    "fields": {
      "course": 2,
      "num": 4,
      "title": "Lesson 4",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 25,
    "fields": {
      "course": 2,
      "num": 5,
      "title": "Lesson 5",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",

```

```

    "pk": 26,
    "fields": {
      "course": 2,
      "num": 6,
      "title": "Lesson 6",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 27,
    "fields": {
      "course": 2,
      "num": 7,
      "title": "Lesson 7",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 28,
    "fields": {
      "course": 2,
      "num": 8,
      "title": "Lesson 8",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 31,
    "fields": {
      "course": 3,
      "num": 1,

```

```

        "title": "Lesson 1",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 32,
    "fields": {
        "course": 3,
        "num": 2,
        "title": "Lesson 2",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 33,
    "fields": {
        "course": 3,
        "num": 3,
        "title": "Lesson 3",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 34,
    "fields": {
        "course": 3,
        "num": 4,
        "title": "Lesson 4",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",

```



```
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 35,
  "fields": {
    "course": 3,
    "num": 5,
    "title": "Lesson 5",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 36,
  "fields": {
    "course": 3,
    "num": 6,
    "title": "Lesson 6",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 37,
  "fields": {
    "course": 3,
    "num": 7,
    "title": "Lesson 7",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
```

```

    "model": "mainapp.lesson",
    "pk": 38,
    "fields": {
        "course": 3,
        "num": 8,
        "title": "Lesson 8",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 41,
    "fields": {
        "course": 4,
        "num": 1,
        "title": "Lesson 1",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 42,
    "fields": {
        "course": 4,
        "num": 2,
        "title": "Lesson 2",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 43,
    "fields": {
        "course": 4,

```

```

        "num": 3,
        "title": "Lesson 3",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 44,
    "fields": {
        "course": 4,
        "num": 4,
        "title": "Lesson 4",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 45,
    "fields": {
        "course": 4,
        "num": 5,
        "title": "Lesson 5",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 46,
    "fields": {
        "course": 4,
        "num": 6,
        "title": "Lesson 6",
        "description": "Описание урока",
        "description_as_markdown": false,

```

```
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 47,
  "fields": {
    "course": 4,
    "num": 7,
    "title": "Lesson 7",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 48,
  "fields": {
    "course": 4,
    "num": 8,
    "title": "Lesson 8",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 51,
  "fields": {
    "course": 5,
    "num": 1,
    "title": "Lesson 1",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
},
```

```

{
  "model": "mainapp.lesson",
  "pk": 52,
  "fields": {
    "course": 5,
    "num": 2,
    "title": "Lesson 2",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 53,
  "fields": {
    "course": 5,
    "num": 3,
    "title": "Lesson 3",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 54,
  "fields": {
    "course": 5,
    "num": 4,
    "title": "Lesson 4",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 55,
  "fields": {

```

```

    "course": 5,
    "num": 5,
    "title": "Lesson 5",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 56,
  "fields": {
    "course": 5,
    "num": 6,
    "title": "Lesson 6",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 57,
  "fields": {
    "course": 5,
    "num": 7,
    "title": "Lesson 7",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 58,
  "fields": {
    "course": 5,
    "num": 8,
    "title": "Lesson 8",
    "description": "Описание урока",

```

```

        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 61,
    "fields": {
        "course": 6,
        "num": 1,
        "title": "Lesson 1",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 62,
    "fields": {
        "course": 6,
        "num": 2,
        "title": "Lesson 2",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 63,
    "fields": {
        "course": 6,
        "num": 3,
        "title": "Lesson 3",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
}

```

```
},
{
  "model": "mainapp.lesson",
  "pk": 64,
  "fields": {
    "course": 6,
    "num": 4,
    "title": "Lesson 4",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 65,
  "fields": {
    "course": 6,
    "num": 5,
    "title": "Lesson 5",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 66,
  "fields": {
    "course": 6,
    "num": 6,
    "title": "Lesson 6",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 67,
```



```

    "fields": {
      "course": 6,
      "num": 7,
      "title": "Lesson 7",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 68,
    "fields": {
      "course": 6,
      "num": 8,
      "title": "Lesson 8",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 71,
    "fields": {
      "course": 7,
      "num": 1,
      "title": "Lesson 1",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 72,
    "fields": {
      "course": 7,
      "num": 2,
      "title": "Lesson 2",

```

```

        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 73,
    "fields": {
        "course": 7,
        "num": 3,
        "title": "Lesson 3",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 74,
    "fields": {
        "course": 7,
        "num": 4,
        "title": "Lesson 4",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 75,
    "fields": {
        "course": 7,
        "num": 5,
        "title": "Lesson 5",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
}

```

```

    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 76,
    "fields": {
      "course": 7,
      "num": 6,
      "title": "Lesson 6",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 77,
    "fields": {
      "course": 7,
      "num": 7,
      "title": "Lesson 7",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",
    "pk": 78,
    "fields": {
      "course": 7,
      "num": 8,
      "title": "Lesson 8",
      "description": "Описание урока",
      "description_as_markdown": false,
      "created": "2021-07-11T12:53:26.832Z",
      "updated": "2021-07-11T12:53:26.833Z"
    }
  },
  {
    "model": "mainapp.lesson",

```

```
"pk": 81,
"fields": {
  "course": 8,
  "num": 1,
  "title": "Lesson 1",
  "description": "Описание урока",
  "description_as_markdown": false,
  "created": "2021-07-11T12:53:26.832Z",
  "updated": "2021-07-11T12:53:26.833Z"
}
},
{
  "model": "mainapp.lesson",
  "pk": 82,
  "fields": {
    "course": 8,
    "num": 2,
    "title": "Lesson 2",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 83,
  "fields": {
    "course": 8,
    "num": 3,
    "title": "Lesson 3",
    "description": "Описание урока",
    "description_as_markdown": false,
    "created": "2021-07-11T12:53:26.832Z",
    "updated": "2021-07-11T12:53:26.833Z"
  }
},
{
  "model": "mainapp.lesson",
  "pk": 84,
  "fields": {
    "course": 8,
    "num": 4,
```

```

        "title": "Lesson 4",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 85,
    "fields": {
        "course": 8,
        "num": 5,
        "title": "Lesson 5",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 86,
    "fields": {
        "course": 8,
        "num": 6,
        "title": "Lesson 6",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 87,
    "fields": {
        "course": 8,
        "num": 7,
        "title": "Lesson 7",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",

```

```

        "updated": "2021-07-11T12:53:26.833Z"
    }
},
{
    "model": "mainapp.lesson",
    "pk": 88,
    "fields": {
        "course": 8,
        "num": 8,
        "title": "Lesson 8",
        "description": "Описание урока",
        "description_as_markdown": false,
        "created": "2021-07-11T12:53:26.832Z",
        "updated": "2021-07-11T12:53:26.833Z"
    }
}
]

```

Фикстура преподавателей

mainapp/fixtures/004_teachers.json

```

[
  {
    "model": "mainapp.courseteachers",
    "pk": 1,
    "fields": {
        "name_first": "Альфред",
        "name_second": "Нуцубидзе",
        "day_birth": "1990-07-10",
        "course": [
            1,
            3
        ]
    }
},
{
    "model": "mainapp.courseteachers",
    "pk": 2,
    "fields": {
        "name_first": "Роман",

```

```

        "name_second": "Доржинов",
        "day_birth": "1988-02-04",
        "course": [
            2,
            4
        ]
    },
    {
        "model": "mainapp.courseteachers",
        "pk": 3,
        "fields": {
            "name_first": "Ярослав",
            "name_second": "Конягин",
            "day_birth": "1981-12-08",
            "course": [
                3,
                5
            ]
        }
    },
    {
        "model": "mainapp.courseteachers",
        "pk": 4,
        "fields": {
            "name_first": "Автандил",
            "name_second": "Наварский",
            "day_birth": "1983-05-16",
            "course": [
                4,
                6
            ]
        }
    },
    {
        "model": "mainapp.courseteachers",
        "pk": 5,
        "fields": {
            "name_first": "Роза",
            "name_second": "Уланова",
            "day_birth": "1986-05-09",

```

```
    "course": [
        5,
        7
    ]
}
},
{
    "model": "mainapp.courseteachers",
    "pk": 6,
    "fields": {
        "name_first": "Бронислава",
        "name_second": "Алиева",
        "day_birth": "1971-01-07",
        "course": [
            6,
            8
        ]
    }
},
{
    "model": "mainapp.courseteachers",
    "pk": 7,
    "fields": {
        "name_first": "Диана",
        "name_second": "Попова",
        "day_birth": "1990-08-25",
        "course": [
            1,
            8
        ]
    }
}
]
```

[Вернуться к теме.](#)

Приложение В. Вывод данных из базы на страницы

Ниже — полное содержимое файлов.

Модели

mainapp/models.py

```
from django.db import models

class News(models.Model):
    title = models.CharField(max_length=256, verbose_name="Title")
    preamble = models.CharField(max_length=1024, verbose_name="Preamble")
    body = models.TextField(blank=True, null=True, verbose_name="Body")
    body_as_markdown = models.BooleanField(
        default=False, verbose_name="As markdown"
    )
    created = models.DateTimeField(
        auto_now_add=True, verbose_name="Created", editable=False
    )
    updated = models.DateTimeField(
        auto_now=True, verbose_name="Edited", editable=False
    )
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return f"{self.pk} {self.title}"

    def delete(self, *args):
        self.deleted = True
        self.save()

class CoursesManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(deleted=False)

class Courses(models.Model):
    objects = CoursesManager()
```

```

name = models.CharField(max_length=256, verbose_name="Name")
description = models.TextField(
    verbose_name="Description", blank=True, null=True
)
description_as_markdown = models.BooleanField(
    verbose_name="As markdown", default=False
)
cost = models.DecimalField(
    max_digits=8, decimal_places=2, verbose_name="Cost", default=0
)
cover = models.CharField(
    max_length=25, default="no_image.svg", verbose_name="Cover"
)
created = models.DateTimeField(auto_now_add=True, verbose_name="Created")
updated = models.DateTimeField(auto_now=True, verbose_name="Edited")
deleted = models.BooleanField(default=False)

def __str__(self) -> str:
    return f"{self.pk} {self.name}"

def delete(self, *args):
    self.deleted = True
    self.save()

class Lesson(models.Model):
    course = models.ForeignKey(Courses, on_delete=models.CASCADE)
    num = models.PositiveIntegerField(verbose_name="Lesson number")
    title = models.CharField(max_length=256, verbose_name="Name")
    description = models.TextField(
        verbose_name="Description", blank=True, null=True
    )
    description_as_markdown = models.BooleanField(
        verbose_name="As markdown", default=False
    )
    created = models.DateTimeField(
        auto_now_add=True, verbose_name="Created", editable=False
    )
    updated = models.DateTimeField(
        auto_now=True, verbose_name="Edited", editable=False
    )
    deleted = models.BooleanField(default=False)

```

```

def __str__(self) -> str:
    return f"{self.course.name} | {self.num} | {self.title}"

def delete(self, *args):
    self.deleted = True
    self.save()

class Meta:
    ordering = ("course", "num")

class CourseTeachers(models.Model):
    course = models.ManyToManyField(Courses)
    name_first = models.CharField(max_length=128, verbose_name="Name")
    name_second = models.CharField(max_length=128, verbose_name="Surname")
    day_birth = models.DateField(verbose_name="Birth date")
    deleted = models.BooleanField(default=False)

    def __str__(self) -> str:
        return "{0:0>3} {1} {2}".format(
            self.pk, self.name_second, self.name_first
        )

    def delete(self, *args):
        self.deleted = True
        self.save()

```

Диспетчер адресов

mainapp/urls.py

```

from django.urls import path

from mainapp import views
from mainapp.apps import MainappConfig

app_name = MainappConfig.name

urlpatterns = [
    path("", views.MainPageView.as_view(), name="main_page"),
    path("news/", views.NewsPageView.as_view(), name="news"),
    path(

```

```

        "news/<int:pk>/", views.NewsPageDetailView.as_view(), name="news_detail"
    ),
    path("courses/", views.CoursesListView.as_view(), name="courses"),
    path(
        "courses/<int:pk>/",
        views.CoursesDetailView.as_view(),
        name="courses_detail",
    ),
    path("contacts/", views.ContactsPageView.as_view(), name="contacts"),
    path("doc_site/", views.DocSitePageView.as_view(), name="doc_site"),
    path("login/", views.LoginPageView.as_view(), name="login"),
]

```

Контроллеры

mainapp/views.py

```

from django.shortcuts import get_object_or_404
from django.views.generic import TemplateView

from mainapp import models as mainapp_models

class MainPageView(TemplateView):
    template_name = "mainapp/index.html"

class NewsPageView(TemplateView):
    template_name = "mainapp/news.html"

    def get_context_data(self, **kwargs):
        # Get all previous data
        context = super().get_context_data(**kwargs)
        # Create your own data
        context["news_qs"] = mainapp_models.News.objects.all()[:5]
        return context

class NewsPageDetailView(TemplateView):
    template_name = "mainapp/news_detail.html"

    def get_context_data(self, pk=None, **kwargs):

```

```
context = super().get_context_data(pk=pk, **kwargs)
context["news_object"] = get_object_or_404(mainapp_models.News, pk=pk)
return context
```

```
class CoursesListView(TemplateView):
    template_name = "mainapp/courses_list.html"

    def get_context_data(self, **kwargs):
        context = super(CoursesListView, self).get_context_data(**kwargs)
        context["objects"] = mainapp_models.Courses.objects.all()[:7]
        return context
```

```
class CoursesDetailView(TemplateView):
    template_name = "mainapp/courses_detail.html"

    def get_context_data(self, pk=None, **kwargs):
        context = super(CoursesDetailView, self).get_context_data(**kwargs)
        context["course_object"] = get_object_or_404(
            mainapp_models.Courses, pk=pk
        )
        context["lessons"] = mainapp_models.Lesson.objects.filter(
            course=context["course_object"]
        )
        context["teachers"] = mainapp_models.CourseTeachers.objects.filter(
            course=context["course_object"]
        )
        return context
```

```
class ContactsPageView(TemplateView):
    template_name = "mainapp/contacts.html"
```

```
class DocSitePageView(TemplateView):
    template_name = "mainapp/doc_site.html"
```

```
class LoginPageView(TemplateView):
    template_name = "mainapp/login.html"
```

Шаблон страницы новостей

mainapp/templates/mainapp/news.html

```
{% extends 'base.html' %}

{% block title %}
Новости
{% endblock title %}

{% block content %}

{% if page_num %}
<div class="card my-2">
  <div class="card-body">
    <p class="card-text">Текущая страница: {{ page_num }}</p>
  </div>
</div>
{% endif %}

<div class="row">

  <div class="col-lg-8">

    {% for item in news_qs %}
    <div class="card my-2">
      <div class="card-body">
        <h5 class="card-title">{{ item.title }}</h5>
        <h6 class="card-subtitle mb-2 text-muted">
          {{ item.created|date:"Y-m-d h-i-s" }}
        </h6>
        <p class="card-text">{{ item.preamble }}</p>
        <a href="{% url 'mainapp:news_detail' pk=item.pk %}"
          class="card-link">Подробнее</a>
      </div>
    </div>
    {% endfor %}

  </div>

  <div class="col-lg-4">
    <div class="card my-2 sticky-top">
      <div class="card-header">
        Фильтры
```

```

</div>
<div class="card-body">
  <form action="">
    <div class="input-group mb-3">
      <div class="input-group-prepend">
        <span class="input-group-text" id="basic-addon1">Дата от...</span>
      </div>
      <input type="date" class="form-control" name="dateFrom">
    </div>
    <div class="input-group mb-3">
      <div class="input-group-prepend">
        <span class="input-group-text" id="basic-addon1">Дата до...</span>
      </div>
      <input type="date" class="form-control" name="dateTo">
    </div>

    <button type="submit"
      class="btn btn-primary btn-block">Фильтровать</button>
  </form>
</div>
</div>
</div>

<div class="row justify-content-center align-items-center mt-3 mb-n3">
  <nav aria-label="Page navigation example">
    <ul class="pagination">
      <li class="page-item"><a class="page-link" href="#">Previous</a></li>
      <li class="page-item"><a class="page-link" href="#">1</a></li>
      <li class="page-item"><a class="page-link" href="#">2</a></li>
      {# <li class="page-item"><a class="page-link" href="#">3</a></li> #}
      <li class="page-item"><a class="page-link" href="#">Next</a></li>
    </ul>
  </nav>
</div>

{% endblock content %}

```

Шаблон списка курсов

mainapp/templates/mainapp/courses_list.html

```

{% extends 'base.html' %}
{% load static %}

{% block title %}
Курсы
{% endblock title %}

{% block content %}

<div class="row justify-content-center mt-3">

    {% for item in objects %}
    <div class="card col-sm-12 col-md-6 col-lg-4 border-0">
        <a href="{% url 'mainapp:courses_detail' pk=item.pk %}">
            
            <div class="card-body">
                <h5 class="card-title text-center">{{ item.name }}</h5>
            </div>
        </a>
    </div>
    {% endfor %}

</div>

<div class="row justify-content-center align-items-center mt-3 mb-n3">
    <nav aria-label="Page navigation example">
        <ul class="pagination">
            <li class="page-item"><a class="page-link" href="#">Previous</a></li>
            <li class="page-item"><a class="page-link" href="#">1</a></li>
            <li class="page-item"><a class="page-link" href="#">2</a></li>
            <li class="page-item"><a class="page-link" href="#">3</a></li>
            <li class="page-item"><a class="page-link" href="#">Next</a></li>
        </ul>
    </nav>
</div>

{% endblock content %}

```

Шаблон детального отображения курса

mainapp/templates/mainapp/courses_detail.html


```

{% extends 'base.html' %}
{% load static markdownify %}

{% block title %}
{{ course_object.name }}
{% endblock title %}

{% block content %}
<h1 class="mt-2">
    {{ course_object.name }}
</h1>
<div class="row justify-content-center">
    <div class="col-sm-12 col-md-6 col-lg-8 order-2 order-md-1">
        <div class="row h-100">
            <div class="col-12">
                {% if course_object.description_as_markdown %}
                {{ course_object.description|markdownify }}
                {% else %}
                {{ course_object.description }}
                {% endif %}
            </div>
            <div class="col-12 align-self-end">
                <p>
                    <small>Преподаватели</small><br>
                    {% for item in teachers %}
                    {{ item.name_second }} {{ item.name_first }}{% if
forloop.last %}{% else %},{% endif %}
                    {% endfor %}
                </p>
            </div>
        </div>
    </div>
    <div class="col-sm-12 col-md-6 col-lg-4 order-1 order-md-2">
        <div class="row h-100">
            <div class="col-12 align-self-start">
                
            </div>
            <div class="col-12 align-self-end d-none d-md-block">
                <p class="text-center mt-2">
                    <small class="text-uppercase">цена</small><br>
                    <span style="font-size: 30pt;">
                        {{ course_object.cost }}
                    </span><br>
                </p>
            </div>
        </div>
    </div>
</div>

```

```

        <a href="#" class="btn btn-outline-primary btn-block"
            role="button">Купить</a>

    </p>
</div>
</div>
</div>
<div class="col-sm-12 d-md-none order-3 order-sm-3">
    <p class="text-center">
        <small class="text-uppercase">цена</small><br>
        <span style="font-size: 30pt;">
            {{ course_object.cost }}
        </span><br>
        <a href="#" class="btn btn-outline-primary btn-block"
            role="button">Купить</a>
    </p>
</div>
</div>
<div class="accordion" id="accordionLessons">

    {% for item in lessons %}
    <div class="card">
        <div class="card-header" id="headLessonPk{{ item.pk }}">
            <h2 class="mb-0">
                <button class="btn btn-link btn-block text-left" type="button"
                    data-toggle="collapse" data-target="#collapse{{ item.pk }}"
                    aria-expanded="true" aria-controls="collapse{{ item.pk }}">
                        {{ item.course.name }} - {{ item.title }}
                </button>
            </h2>
        </div>

        <div id="collapse{{ item.pk }}" class="collapse"
            aria-labelledby="headLessonPk{{ item.pk }}"
            data-parent="#accordionLessons">
            <div class="card-body">
                <div class="row justify-content-center">
                    <div class="col-12 text-center">
                        
                    </div>
                    <div class="col-12">
                        {% if item.description_as_markdown %}
                        {{ item.description|markdownify }}

```

```

        {% else %}
        {{ item.description }}
        {% endif %}
    </div>
</div>
</div>
</div>
</div>
{% endfor %}

</div>
{% endblock content %}

```

Шаблон детального отображения новости

mainapp/templates/mainapp/news_detail.html

```

{% extends 'base.html' %}

{% block title %}
{{ news_object.title|default:"Новости" }}
{% endblock title %}

{% block content %}
<div class="row justify-content-center pt-3">
    <div class="col-md-12 col-lg-8">
        <div class="card shadow">
            <div class="card-body">
                <h5 class="card-title">{{ news_object.title }}</h5>
                <p class="card-text">
                    {{ news_object.body }}
                </p>
                <a href="{% url 'mainapp:news' %}"
                    class="btn btn-primary">Назад</a>
            </div>
        </div>
    </div>
</div>
{% endblock content %}

```

Базовый шаблон

templates/base.html

```
{% load static %}

<!doctype html>
<html lang="ru">

<head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport"
        content="width=device-width, initial-scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href="{% static 'css/bootstrap.min.css' %}">

    <!-- ChartJS -->
    <link rel="stylesheet" href="{% static 'css/Chart.min.css' %}">

    <!-- FontAwesome -->
    <link rel="stylesheet" href="{% static 'css/fontawesome.all.min.css' %}">

    <title>
        {% block title %}
        Welcome to Braniac!
        {% endblock title %}
    </title>
</head>

<body>

    {% include 'includes/main_menu.html' %}

    <div class="container-md">

        {% block content %}

        Lorem ipsum dolor sit amet, consectetur adipisicing elit. Aspernatur
        optio est esse? Veritatis laborum expedita
        quibusdam numquam! Expedita amet nisi dolor similique autem, mollitia
        ipsa et, natus vel explicabo
        reprehenderit?
```

```

{% endblock content %}

</div>

<!-- Footer -->

<div class="container-lg mt-auto">
  <hr>
  <div class="row justify-content-center">
    <div class="col-sm-6 col-md-3 text-center">
      <p>
        <strong>Braniac</strong>
      </p>
      <p>
        <ul class="list-unstyled">
          <li><a href="{% url 'mainapp:main_page' %}">Домашняя</a></li>
          <li><a href="{% url 'mainapp:news' %}">Новости</a></li>
          <li><a href="{% url 'mainapp:login' %}">Войти</a></li>
        </ul>
      </p>
    </div>
    <div class="col-sm-6 col-md-3 text-center">
      <p>
        <strong>Полезное</strong>
      </p>
      <p>
        <ul class="list-unstyled">
          <li><a href="#">Положения &amp; Условия</a></li>
          <li><a href="#">Конфиденциальность &amp; Cookies</a></li>
          <li><a href="#">Документация по API</a></li>
          <li><a href="{% url 'mainapp:doc_site' %}">Документация по сайту</a>
            </li>
        </ul>
      </p>
    </div>
    <div class="col-sm-6 col-md-3 text-center">
      <p>
        <strong>Мы в социальных сетях</strong>
      </p>
      <p>
        <div class="row justify-content-around">
          <div><a href="#"><i class="fab fa-vk fa-2x"></i></a></div>

```

```

        <div><a href="#"><i class="fab fa-facebook-f fa-2x"></i></a>
    </div>
    <div><a href="#"><i class="fab fa-instagram fa-2x"></i></a>
    </div>
    <div><a href="#"><i class="fab fa-pinterest-p fa-2x"></i></a></div>
</div>
</p>
<p>
    <strong>Наше приложение</strong>
</p>
<p>
    <div class="row justify-content-around">
        <div><a href="#"><i class="fab fa-app-store fa-2x"></i></a>
        </div>
        <div><a href="#"><i class="fab fa-google-play fa-2x"></i></a></div>
        <div><a href="#"><i class="fab fa-windows fa-2x"></i></a>
        </div>
    </div>
</p>
</div>
<div class="row justify-content-center">
    <div>
        <p><small>&copy; GeekBrains 2021</small></p>
    </div>
</div>
</div>

<!-- /Footer -->

<!-- JavaScript section -->
<!-- Bootstrap -->
<script src="{% static 'js/jquery-3.6.0.min.js' %}"></script>
<script src="{% static 'js/popper.min.js' %}"></script>
<script src="{% static 'js/bootstrap.min.js' %}"></script>

<!-- ChartJs -->
<script src="{% static 'js/Chart.min.js' %}"></script>

<!-- FontAwesome -->

```

```
<script src="{% static 'js/fontawesome.all.min.js' %}"></script>
```

```
</body>
```

```
</html>
```

Изображение по умолчанию

static/img/no_image.svg

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg xmlns="http://www.w3.org/2000/svg" style="background-color: rgb(255, 255, 255);" xmlns:xlink="http://www.w3.org/1999/xlink" version="1.1" width="1287px" height="860px" viewBox="-0.5 -0.5 1287 860" content="&lt;mxfile host=&quot;app.diagrams.net&quot; modified=&quot;2021-07-10T11:53:54.271Z&quot; agent=&quot;5.0 (Windows)&quot; etag=&quot;Xq7wZiGO59QnJTieiluy&quot; version=&quot;14.8.4&quot; type=&quot;device&quot;&gt;&lt;diagram id=&quot;1W_2InFo7JNA8gz3PVU6&quot; name=&quot;Page-2&quot;&gt;7VZNb6MwEP01HCsBLtnkWJK0vay2bbq77dGLJ2DVYOo4Bfrr14Qxn5GyjXqqVhzieZ55o3nzEHHIMilvFM2T75KBcHyXlQ5Zob7vzb4R81MjVYMsvHkDxIozTOqADX8HBF1E95zBbpCopRSA50MwklkGkR5gVC1ZDNO2Ugy75jTGjm4HbCIqYJL2mzOdNChxe9m3wOMEO/v2IqU2F4FdQpksehBZO2SppNTNKS2XIGrtrCz3m9v1rz/kIaE8nMUvlf32Sl00ZNcfKWknUJDps6kfftCbdxbmQ2Tx+ArBz2u5kljivlGxR7lwVl1Z/ZTcZwxqEtchYZFwDZucRvVtYRxjsESnwkSeOe60ki+wleIqg2QyM2nh1gthIccn61n9GBx7g9JQjnZ1YlKvld/YFmQKWlWmzrLYjaFjLzEsuvV7/hzBpLf7eYBep+i5uOXuhDUH1PYDONundYaMXdv273Sb6Gr9G0yUNrKGh8fcGOlU9YT7OgTP/WBVDqLKRiXXT9itPj/3z11JHdiKZgJg8fhF+5flmdHlXkVw2puaqhj0qbypGXrbDo7s2mIKBNX8bTjDsfljhzvJzXStl+buyGsLd0jRjIlV/ddzTBSMiC5HRI0OE6KDHduxz3eo/ykOPebHqXdbH3rnONT979Av5lATdp/RJr37L0LWfwE=&lt;/diagram&gt;&lt;/mxfile&gt;"><defs/><g><rect x="2" y="2" width="1280" height="853" fill="#e6e6e6" stroke="none" pointer-events="all"/><path d="M 1282 855 L 2 2" fill="none" stroke="#b3b3b3" stroke-width="5" stroke-miterlimit="10" pointer-events="stroke"/><path d="M 2 855 L 1282 2" fill="none" stroke="#b3b3b3" stroke-width="5" stroke-miterlimit="10" pointer-events="stroke"/></g></svg>
```

[Вернуться к теме.](#)